Event Management System

USER
├── Register
├── Login
├── View Events
├── Search Events
├── Filter Events
├── Register for Event
└── View My Registrations

ADMIN
├── Login
├── Create Event
├── Update Event
├── Delete Event
└── View Registrations

Create Project

- Make Folder

mkdir event-management-system
cd event-management-system

- Then create React:

npm create vite@latest client

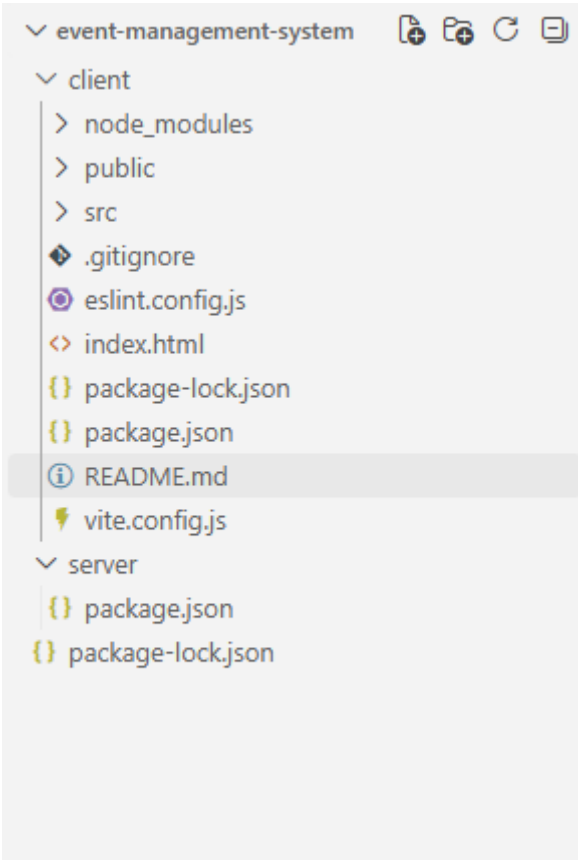
Select a framework: → React
Select a variant: → JavaScript
Select a Linter → ESLint

- Check for installation

Move to client folder
npm install

Go back and make server folder
cd ..
mkdir server
cd server
npm init -y

- Now your VS Code Explorer should look like:



Install Backend Packages

- Inside server/

  npm install express mongoose cors dotenv bcryptjs jsonwebtoken socket.io

  this command installs the common backend dependencies

Package	Why it is needed	Example use
express	Creates the backend/server and API routes	`GET /users`, `POST /events`
mongoose	Connects Node.js to MongoDB and defines database schemas	User, Event, Registration models
cors	Allows your React frontend and backend to communicate when they run on different ports	React `localhost:5173` → API `localhost:5000`

dotenv	Loads secret/configuration values from `.env`	MongoDB URL, JWT secret, port
bcryptjs	Securely hashes passwords before storing them	Hash login password
jsonwebtoken	Creates and verifies JWT tokens for authentication	Login → generate token → access protected API
socket.io	Provides real-time communication between client and server	Live notifications, chat, real-time event updates

- For development:

  npm install -D nodemon

Create Your Folder Structure in VS Code

- Create the Backend Structure

  cd server

  mkdir config controllers middleware models routes utils OR

  mkdir config, controllers, middleware, models, routes, utils (on Powershell)

- Create the files:

  type nul > server.js

  type nul > .env

  type nul > .gitignore

  type nul > config\db.js

  type nul > controllers\authController.js

  type nul > controllers\eventController.js

type nul > middleware\authMiddleware.js

type nul > middleware\errorMiddleware.js

type nul > models\User.js

type nul > models\Event.js

type nul > models\Registration.js

type nul > routes\authRoutes.js

type nul > routes\eventRoutes.js

type nul > routes\registrationRoutes.js

type nul > utils\generateToken.js

OR (for powershell)

New-Item server.js -ItemType File

New-Item .env -ItemType File

New-Item config\db.js -ItemType File

New-Item middleware\authMiddleware.js -ItemType File

New-Item utils\generateToken.js -ItemType File

- Create the Frontend Structure

  cd ..

  cd client

- Create the folders:

  mkdir src\assets src\components src\pages src\context src\hooks src\services src\utils

OR mkdir src\assets, src\components, src\pages, src\context, src\hooks, src\services, src\utils

- Create the component files:

  type nul > src\components\Navbar.jsx

  type nul > src\components\EventCard.jsx

  type nul > src\components\Button.jsx

  type nul > src\components\Modal.jsx

  type nul > src\components\Loading.jsx

  OR

New-Item src\components\Navbar.jsx -ItemType File

New-Item src\components\EventCard.jsx -ItemType File

New-Item src\components\Button.jsx -ItemType File

New-Item src\components\Modal.jsx -ItemType File

New-Item src\components\Loading.jsx -ItemType File

- Create the page files:

  type nul > src\pages\Home.jsx

  type nul > src\pages\Events.jsx

  type nul > src\pages\Login.jsx

  type nul > src\pages\Register.jsx

  type nul > src\pages\MyRegistrations.jsx

  type nul > src\pages\AdminDashboard.jsx

  OR

  New-Item src\pages\Home.jsx -ItemType File

  New-Item src\pages\Events.jsx -ItemType File

  New-Item src\pages\Login.jsx -ItemType File

  New-Item src\pages\Register.jsx -ItemType File

  New-Item src\pages\MyRegistrations.jsx -ItemType File

New-Item src\pages\AdminDashboard.jsx -ItemType File

- Create Context, Hook and API files:

type nul > src\context\AuthContext.jsx

type nul > src\hooks\useAuth.js

type nul > src\services\api.js

OR

New-Item src\context\AuthContext.jsx -ItemType File

New-Item src\hooks\useAuth.js -ItemType File

New-Item src\services\api.js -ItemType File

Finally you should see

E:\event-management-system

```
│
├── client
│   ├── src
│   │   ├── assets
│   │   ├── components
│   │   │   ├── Button.jsx
│   │   │   ├── EventCard.jsx
│   │   │   ├── Loading.jsx
│   │   │   ├── Modal.jsx
│   │   │   └── Navbar.jsx
│   │   │
│   │   ├── context
│   │   │   └── AuthContext.jsx
```

```
│   │   │
│   │   ├── hooks
│   │   │   └── useAuth.js
│   │   │
│   │   ├── pages
│   │   │   ├── AdminDashboard.jsx
│   │   │   ├── Events.jsx
│   │   │   ├── Home.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── MyRegistrations.jsx
│   │   │   └── Register.jsx
│   │   │
│   │   ├── services
│   │   │   └── api.js
│   │   │
│   │   └── utils
│   │
│   └── package.json
│
└── server
    ├── config
    │   └── db.js
    ├── controllers
    │   ├── authController.js
    │   └── eventController.js
    ├── middleware
```

```
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├── models
│   ├── Event.js
│   ├── Registration.js
│   └── User.js
├── routes
│   ├── authRoutes.js
│   ├── eventRoutes.js
│   └── registrationRoutes.js
├── utils
│   └── generateToken.js
├── .env
├── .gitignore
├── package.json
└── server.js
```

Tailwind CSS Setup

- Install Tailwind CSS

  cd E:\event-management-system\client

  npm install tailwindcss @tailwindcss/vite

- Configure Vite

  Open:

  client/vite.config.js

Replace its contents with:

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

- Add Tailwind to your CSS

  Open:

  client/src/index.css

  Put

  @import "tailwindcss";

- Test Tailwind

  Open:

  client/src/App.jsx

  Use:

```
function App() {
  return (
    <div className="min-h-screen bg-blue-600 flex items-center justify-center">
      <h1 className="text-4xl font-bold text-white">
        Tailwind CSS is Working!
      </h1>
```

```
    </div>
  )
}

export default App
```

- Start the application

  npm run dev

  Then open the URL Vite gives you, usually:

  http://localhost:5173

TRY

- Other text sizes - text-xs text-sm text-base text-lg text-xl text-2xl text-3xl text-4xl text-5xl text-6xl text-7xl
- Change bg colour: - bg-{color}-{shade}
- Text color: - text-black, text-red-500
- Font weight: - font-light font-normal font-medium font-semibold font-bold font-extrabold font-black
- Alignment:- text-left text-center text-right
- Padding:- p-{value} px-4 py-4 pt-4 pb-4 pl-4 pr-4
- Margin:-
  m-4 mt-4 mb-4 mx-auto
- Width and Height:- w-64 w-full w-1/2 h-32 h-screen min-h-screen
- Border:- border border-2 border-4 border-blue-500
- Rounded corner:- rounded rounded-md rounded-lg rounded-xl rounded-2xl rounded-full
- Shadow:- shadow shadow-md shadow-lg shadow-xl shadow-2xl
- Hover effect:-hover:bg-blue-800 hover:bg-red-600 hover:text-yellow-300 hover:scale-110

Build a **small static Event Management homepage**

Navbar

↓
Hero Section
↓
Event Cards
↓
Register Buttons

Open:

client/src/components/Navbar.jsx

Paste:

```
function Navbar() {
  return (
    <nav className="bg-slate-900 text-white">
      <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

        <h1 className="text-2xl font-bold">
          EventHub
        </h1>

        <div className="flex gap-6">
          <a href="#" className="hover:text-indigo-400">
            Home
          </a>

          <a href="#" className="hover:text-indigo-400">
            Events
          </a>

          <a href="#" className="hover:text-indigo-400">
            Login
          </a>
        </div>

      </div>
    </nav>
  );
}

export default Navbar;
```

Open:

client/src/App.jsx

Paste:

```
import Navbar from "./components/Navbar";

function App() {
  return (
    <div>
      <Navbar />
    </div>
  );
}

export default App;
```

Also note

Index.css

- In the global CSS, I added a navbar-specific rule so the `h1` inside the nav does not inherit the dark page heading color:

```
nav h1 { color: #fff; margin: 0; }
```

Check the browser

You should see:

```
┌──────────────────────────────────────────────────────────┐
│ EventHub            Home  Events  Login    │
└──────────────────────────────────────────────────────────┘
```

Now create:

client/src/pages/Hooĺppppppppp me.jsx

```
function Home() {
```

```
  return (
    <section className="bg-slate-100 px-6 py-20 text-center">
      <h1 className="text-4xl font-bold text-slate-900 md:text-5xl">
        Discover Amazing Events
      </h1>

      <p className="mx-auto mt-4 max-w-2xl text-lg text-slate-600">
        Find exciting events, connect with people, and register for
        experiences that interest you.
      </p>

      <button className="mt-8 rounded-lg bg-indigo-600 px-6 py-3 font-semibold
text-white transition hover:bg-indigo-700">
        Explore Events
      </button>
    </section>
  );
}

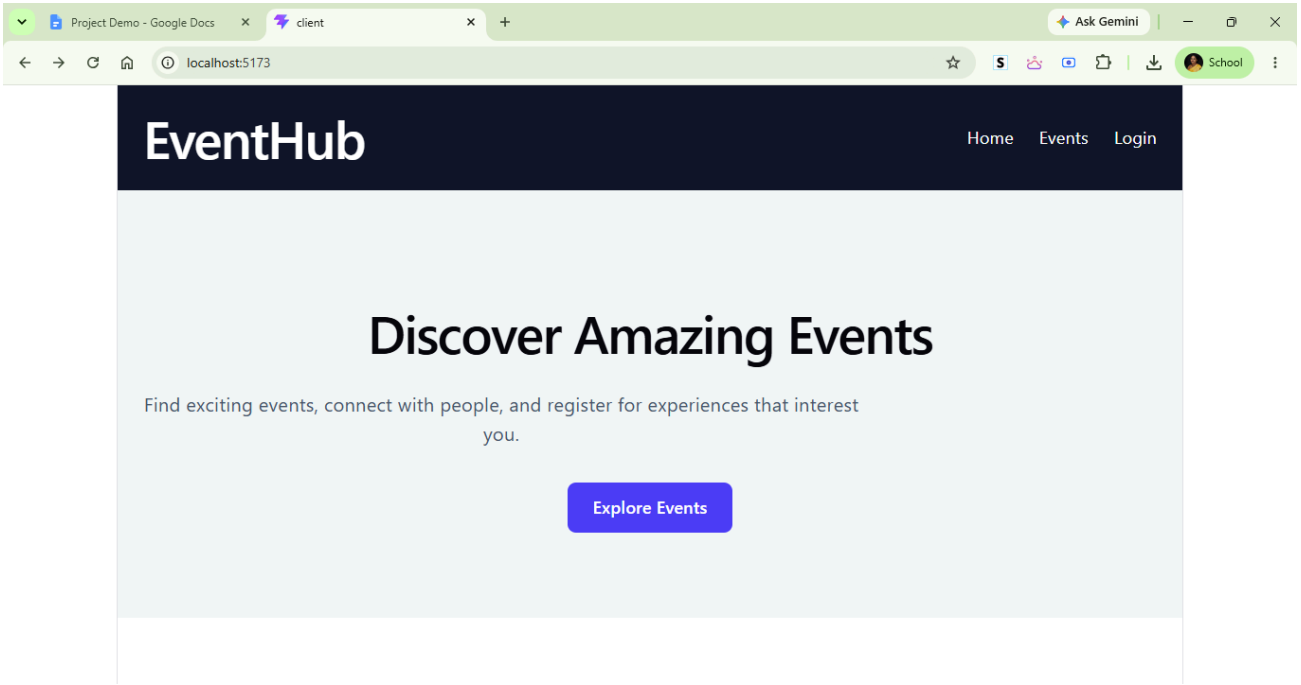
export default Home;
```

Change App.jsx:

```
import Navbar from "./components/Navbar";
import Home from "./pages/Home";

function App() {
  return (
    <div>
      <Navbar />
      <Home />
    </div>
  );
}

export default App;
```



Add Event Card

Open:

client/src/components/EventCard.jsx

```
function EventCard() {
  return (
    <div className="overflow-hidden rounded-xl bg-white shadow-md transition hover:-translate-y-1 hover:shadow-xl">

      <div className="h-40 bg-indigo-500"></div>

      <div className="p-6">
        <span className="rounded-full bg-green-100 px-3 py-1 text-sm text-green-700">
          Open
        </span>

        <h2 className="mt-4 text-xl font-bold text-slate-800">
          React Workshop
        </h2>

        <p className="mt-2 text-slate-600">
          Learn React and build modern web applications.
        </p>

        <p className="mt-4 text-sm text-slate-500">
```

```
          25 August 2026
        </p>

      </div>
    </div>
  );
}

export default EventCard;
```

Add Event Cards to Home

Open:

client/src/pages/Home.jsx

```
import EventCard from "../components/EventCard";

function Home() {
  return (
    <div>
      {/* Hero Section */}
      <section className="bg-slate-100 px-6 py-20 text-center">
        <h1 className="text-4xl font-bold text-slate-900 md:text-5xl">
          Discover Amazing Events
        </h1>

        <p className="mx-auto mt-4 max-w-2xl text-lg text-slate-600">
          Find exciting events, connect with people, and register for
          experiences that interest you.
        </p>

        <button className="mt-8 rounded-lg bg-indigo-600 px-6 py-3 font-semibold text-white
transition hover:bg-indigo-700">
          Explore Events
        </button>
      </section>

      {/* Events Section */}
      <section className="bg-white px-6 py-16">
        <div className="mx-auto max-w-7xl">

          <h2 className="text-3xl font-bold text-slate-900">
            Upcoming Events
          </h2>
```

```
        <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
          <EventCard />
          <EventCard />
          <EventCard />
        </div>

      </div>
    </section>
  </div>
 );
}

export default Home;
```

TRY

- flex
- Items-center
- Justify-between
- flex-row
- flex-col
- md:* lg:* etc
- grid grid-cols-*
- md:text-5xl md:bg-blue-100 md:p-10 md:flex-row md:grid-cols-2 md:hidden
- Transition

    In:

    client/src/components/EventCard.jsx

    inside the `p-6` div, after the date, add:

    ```
    <button className="mt-6 rounded-lg bg-indigo-600 px-4 py-3 font-semibold
    text-white transition hover:bg-indigo-700">
      Register Now
    </button>
    ```

- w-full md:w-auto

React

As of now we have updated Navbar, Eventcard, Home

Props

Change `EventCard.jsx` to:

```
function EventCard({ title, description }) {
  return (
    <div className="rounded-xl border bg-white p-6 shadow-lg">
      <h3 className="text-2xl font-bold text-slate-900">
        {title}
      </h3>

      <p className="mt-2 text-slate-600">
        {description}
      </p>

      <button className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white hover:bg-indigo-700">
        Register
      </button>
    </div>
  );
}

export default EventCard;
```

Now change your `Home.jsx`:

```
<EventCard
  title="Tech Fest 2026"
  description="A technology event for students."
/>

<EventCard
  title="React Workshop"
  description="Learn React from basics to advanced."
/>

<EventCard
  title="AI Conference"
  description="Explore the latest developments in AI."
/>
```

Add more props

Change:

function EventCard({ title, description }) {

to:

```
function EventCard({
  title,
  description,
  date,
  location
}) {
```

Then inside the card:

```
<p className="mt-2 text-sm text-slate-500">
  📅 {date}
</p>

<p className="mt-1 text-sm text-slate-500">
  📍 {location}
</p>
```

Now in `Home.jsx`:

```
<EventCard
  title="Tech Fest 2026"
  description="A technology event for students."
  date="20 September 2026"
  location="Mumbai"
/>
```

Another:

```
<EventCard
  title="React Workshop"
  description="Learn React from basics to advanced."
  date="25 September 2026"
  location="Pune"
/>
```

In home,jsx
Change Hero section

```
<h1 className="text-2xl md:text-6xl font-bold text-red-500">
  Welcome to EventHub
</h1>
<p>
  Discover and register for exciting events.
</p>
```

Event

Add event to the Button

Inside `EventCard`:

```
<button
  onClick={() => alert(`Registering for ${title}`)}
  className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white hover:bg-indigo-700"
>
  Register
</button>
```

usestate()

Open EventCard.jsx
Add on top

```
import { useState } from "react";
```

Create the registered state

First in the EventCard

```
const [registered, setRegistered] = useState(false);
```

Replace your old alert()

Change it to

```
<button

  onClick={() => setRegistered(true)}

  className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white"

>
```

```
  Register
</button>
```

To show success message

Put the success message **below the Register button**.

For example:

```
<button
  onClick={() => setRegistered(true)}
  className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white"
>
  Register
</button>

{registered && (
  <p className="mt-3 font-semibold text-green-600">
    Registration successful!
  </p>
)}
```

Complete `EventCard.jsx`

```
import { useState } from "react";

function EventCard({ title, date, location }) {
  const [registered, setRegistered] = useState(false);

  return (
    <div className="rounded-xl border bg-white p-6 shadow-md">

      <h2 className="text-xl font-bold text-slate-900">
        {title}
      </h2>
```

```
      <p className="mt-2 text-slate-600">
        Date: {date}
      </p>

      <p className="text-slate-600">
        Location: {location}
      </p>

      <button
        onClick={() => setRegistered(true)}
        className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white transition
hover:bg-indigo-700"
      >
        Register
      </button>

      {registered && (
        <p className="mt-3 font-semibold text-green-600">
          Registration successful!
        </p>
      )}

    </div>
  );

}
```

Conditional Rendering

Replace the button with:

```
{registered ? (
  <p className="mt-4 font-semibold text-green-600">
    Successfully Registered!
  </p>
) : (
  <button
    onClick={() => setRegistered(true)}
    className="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-white"
  >
    Register
  </button>
)}
```

Lists / map()

Home has:

```
<EventCard />
<EventCard />
<EventCard />
```

Eventually we don't want to manually write every event.

Create an array:

Your `events` array becomes:

```
const events = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "15 September 2026",
    location: "Mumbai",
  },
  {
    id: 2,
    title: "React Workshop",
    description: "Learn React from basics to advanced.",
    date: "20 September 2026",
    location: "Pune",
  },
  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest AI technologies.",
    date: "25 September 2026",
    location: "Bengaluru",
  },
];
```

Then update y

Then:

```
<div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
  {events.map((event) => (
    <EventCard
      key={event.id}
```

```
      title={event.title}
      description={event.description}
    />
  ))}
</div>
```

Dynamic page rather than Static page

Create `Events.jsx`

Open

client/src/pages/Events.jsx

Start with this code:

```
import EventCard from "../components/EventCard";

const events = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "20 September 2026",
    location: "Mumbai",
  },
  {
    id: 2,
    title: "React Workshop",
    description: "Learn React from basics to advanced.",
    date: "25 September 2026",
    location: "Pune",
  },
  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest developments in AI.",
    date: "30 September 2026",
    location: "Bangalore",
  },
];

function Events() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">
```

```
      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          Upcoming Events
        </h1>

        <p className="mt-2 text-slate-600">
          Explore our upcoming events and register for the ones you like.
        </p>

        <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
          {events.map((event) => (
            <EventCard
              key={event.id}
              title={event.title}
              description={event.description}
              date={event.date}
              location={event.location}
            />
          ))}
        </div>

      </div>
    </div>
  );
}

export default Events;
```

Router

Your `Home.jsx` should start like this:

```
function Home() {
  return (
    <div className="min-h-screen">

      {/* Hero Section */}
      <section className="bg-slate-100 px-6 py-20 text-center">
        <h1 className="text-4xl font-bold text-slate-900 md:text-6xl">
          Discover Amazing Events
        </h1>

        <p className="mx-auto mt-4 max-w-2xl text-lg text-slate-600">
          Find exciting events, connect with people, and register for
          experiences that interest you.
```

```
        </p>

        <button className="mt-8 rounded-lg bg-indigo-600 px-6 py-3 font-semibold text-white
transition hover:bg-indigo-700">
          Explore Events
        </button>
      </section>

      {/* Introduction Section */}
      <section className="bg-white px-6 py-16 text-center">
        <h2 className="text-3xl font-bold text-slate-900">
          Find Your Next Experience
        </h2>

        <p className="mx-auto mt-4 max-w-2xl text-slate-600">
          Explore technology workshops, conferences, festivals, and
          other exciting events happening near you.
        </p>
      </section>

    </div>
  );
}

export default Home;
```

Add routs to App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Navbar from "./components/Navbar";
import Home from "./pages/Home";
import Events from "./pages/Events";

function App() {
  return (
    <BrowserRouter>
      <Navbar />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/events" element={<Events />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```
export default App;
```

Update Navbar

As of now it is

```
function Navbar() {
  return (
    <nav className="bg-slate-900 text-white">
      <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

        <h1 className="text-2xl font-bold">
          EventHub
        </h1>

        <div className="flex gap-6">
          <a href="#">Home</a>
          <a href="#">Events</a>
          <a href="#">Login</a>
        </div>

      </div>
    </nav>
  );
}

export default Navbar;
```

Add routes

```
import { Link } from "react-router-dom";
```

and change the links to:

```
<Link to="/">Home</Link>
<Link to="/events">Events</Link>
```

Form

To begin with

App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Navbar from "./components/Navbar";
```

```
import Home from "./pages/Home";
import Events from "./pages/Events";
import Login from "./pages/Login";
import Register from "./pages/Register";
import MyRegistrations from "./pages/MyRegistrations";
import AdminDashboard from "./pages/AdminDashboard";

function App() {
  return (
    <BrowserRouter>
      <Navbar />

      <Routes>
        <Route path="/" element={<Home />} />

        <Route path="/events" element={<Events />} />

        <Route path="/login" element={<Login />} />

        <Route path="/register" element={<Register />} />

        <Route
          path="/my-registrations"
          element={<MyRegistrations />}
        />

        <Route
          path="/admin"
          element={<AdminDashboard />}
        />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Navbar.jsx

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav className="bg-slate-900 text-white">
      <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

        <h1 className="text-2xl font-bold">
          EventHub
        </h1>

        <div className="flex gap-6">

          <Link
            to="/"
```

```
            className="transition hover:text-indigo-400"
          >
            Home
          </Link>

          <Link
            to="/events"
            className="transition hover:text-indigo-400"
          >
            Events
          </Link>

          <Link
            to="/login"
            className="transition hover:text-indigo-400"
          >
            Login
          </Link>

          <Link
            to="/register"
            className="transition hover:text-indigo-400"
          >
            Register
          </Link>

        </div>
      </div>
    </nav>
  );
}
export default Navbar;
```

Home.jsx

```
function Home() {
  return (
    <div className="min-h-screen">

      {/* Hero Section */}
      <section className="bg-slate-100 px-6 py-20 text-center">

        <h1 className="text-4xl font-bold text-slate-900 md:text-6xl">
          Discover Amazing Events
        </h1>

        <p className="mx-auto mt-4 max-w-2xl text-lg text-slate-600">
          Find exciting events, connect with people, and
          register for experiences that interest you.
        </p>
```

```
        <button
          className="mt-8 rounded-lg bg-indigo-600 px-6 py-3
                     font-semibold text-white transition
                     hover:bg-indigo-700"
        >
          Explore Events
        </button>

      </section>

      {/* Introduction Section */}
      <section className="bg-white px-6 py-16 text-center">

        <h2 className="text-3xl font-bold text-slate-900">
          Find Your Next Experience
        </h2>

        <p className="mx-auto mt-4 max-w-2xl text-slate-600">
          Explore technology workshops, conferences,
          festivals, and other exciting events happening
          near you.
        </p>

      </section>

    </div>
  );
}
export default Home;
```

EventCard.jsx

```
import { useState } from "react";

function EventCard({
  title,
  description,
  date,
  location,
}) {
  const [registered, setRegistered] = useState(false);

  return (
    <div className="rounded-xl border bg-white p-6 shadow-md">

      <h2 className="text-xl font-bold text-slate-900">
        {title}
      </h2>
```

```
      <p className="mt-2 text-slate-600">
        {description}
      </p>

      <p className="mt-3 text-slate-600">
        Date: {date}
      </p>

      <p className="text-slate-600">
        Location: {location}
      </p>

      {registered ? (
        <p className="mt-4 font-semibold text-green-600">
          Successfully Registered!
        </p>
      ) : (
        <button
          onClick={() => setRegistered(true)}
          className="mt-4 rounded-lg bg-indigo-600 px-4 py-2
                     text-white transition
                     hover:bg-indigo-700"
        >
          Register
        </button>
      )}

    </div>
  );
}

export default EventCard;
```

Events.jsx

```
import EventCard from "../components/EventCard";

const events = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "20 September 2026",
    location: "Mumbai",
  },
  {
```

```
    id: 2,
    title: "React Workshop",
    description: "Learn React from basics to advanced.",
    date: "25 September 2026",
    location: "Pune",
  },
  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest developments in AI.",
    date: "30 September 2026",
    location: "Bangalore",
  },
];

function Events() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          Upcoming Events
        </h1>

        <p className="mt-2 text-slate-600">
          Explore our upcoming events and register
          for the ones you like.
        </p>

        <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">

          {events.map((event) => (
            <EventCard
              key={event.id}
              title={event.title}
              description={event.description}
              date={event.date}
              location={event.location}
            />
          ))}

        </div>

      </div>

    </div>
  );
```

```
}

export default Events;
```

Add function Registration Form

```
Register() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Create Account
        </h1>

        <p className="mt-2 text-slate-600">
          Register to start exploring events.
        </p>

        <form className="mt-6 space-y-4">

          <div>
            <label className="mb-1 block font-medium">
              Name
            </label>

            <input
              type="text"
              placeholder="Enter your name"
              className="w-full rounded-lg border px-4 py-2
                        outline-none focus:ring-2
                        focus:ring-indigo-500"
            />
          </div>

          <div>
            <label className="mb-1 block font-medium">
              Email
            </label>

            <input
              type="email"
              placeholder="Enter your email"
              className="w-full rounded-lg border px-4 py-2
                        outline-none focus:ring-2
                        focus:ring-indigo-500"
            />
          </div>
```

```
        <div>
          <label className="mb-1 block font-medium">
            Password
          </label>

          <input
            type="password"
            placeholder="Enter your password"
            className="w-full rounded-lg border px-4 py-2
                       outline-none focus:ring-2
                       focus:ring-indigo-500"
          />
        </div>

        <button
          type="submit"
          className="w-full rounded-lg bg-indigo-600
                     px-4 py-3 font-semibold text-white
                     transition hover:bg-indigo-700"
        >
          Register
        </button>

      </form>

    </div>

  </div>
 );
}

export default Register;
```

Add function Login Form

```
 Login() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Login
        </h1>

        <p className="mt-2 text-slate-600">
          Login to manage your event registrations.
```

```
        </p>

        <form className="mt-6 space-y-4">

          <div>
            <label className="mb-1 block font-medium">
              Email
            </label>

            <input
              type="email"
              placeholder="Enter your email"
              className="w-full rounded-lg border px-4 py-2
                         outline-none focus:ring-2
                         focus:ring-indigo-500"
            />
          </div>

          <div>
            <label className="mb-1 block font-medium">
              Password
            </label>

            <input
              type="password"
              placeholder="Enter your password"
              className="w-full rounded-lg border px-4 py-2
                         outline-none focus:ring-2
                         focus:ring-indigo-500"
            />
          </div>

          <button
            type="submit"
            className="w-full rounded-lg bg-indigo-600
                       px-4 py-3 font-semibold text-white
                       transition hover:bg-indigo-700"
          >
            Login
          </button>

        </form>

      </div>

    </div>
  );
}
```

```
export default Login;
```

Add MyRegistrations.jsx

We need this file because it is already part of our routing.

For now, just give it a placeholder UI:

```
function MyRegistrations() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          My Registrations
        </h1>

        <p className="mt-2 text-slate-600">
          Your registered events will appear here.
        </p>

      </div>

    </div>
  );
}

export default MyRegistrations;
```

Later this page will get its data from the backend.

Add AdminDashboard.jsx

Again, placeholder for now:

```
function AdminDashboard() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          Admin Dashboard
        </h1>

        <p className="mt-2 text-slate-600">
          Manage events and view registrations.
```

```
        </p>

      </div>

    </div>
  );
}

export default AdminDashboard;
```

Form Validation

Make `Register.jsx` functional

Replace your `Register.jsx` with:

```
import { useState } from "react";

function Register() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
  });

  const [message, setMessage] = useState("");

  function handleChange(event) {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value,
    });
  }

  function handleSubmit(event) {
    event.preventDefault();

    if (!formData.name || !formData.email || !formData.password) {
      setMessage("Please fill in all fields.");
      return;
    }

    if (formData.password.length < 6) {
      setMessage("Password must be at least 6 characters.");
      return;
```

```
  }

  setMessage("Registration successful!");
}

return (
  <div className="min-h-screen bg-slate-100 px-6 py-16">
    <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">

      <h1 className="text-3xl font-bold text-slate-900">
        Create Account
      </h1>

      <p className="mt-2 text-slate-600">
        Register to start exploring events.
      </p>

      <form
        onSubmit={handleSubmit}
        className="mt-6 space-y-4"
      >

        {/* Name */}
        <div>
          <label className="mb-1 block font-medium">
            Name
          </label>

          <input
            type="text"
            name="name"
            value={formData.name}
            onChange={handleChange}
            placeholder="Enter your name"
            className="w-full rounded-lg border px-4 py-2
                       outline-none focus:ring-2
                       focus:ring-indigo-500"
          />
        </div>

        {/* Email */}
        <div>
          <label className="mb-1 block font-medium">
            Email
          </label>

          <input
            type="email"
```

```
            name="email"
            value={formData.email}
            onChange={handleChange}
            placeholder="Enter your email"
            className="w-full rounded-lg border px-4 py-2
                       outline-none focus:ring-2
                       focus:ring-indigo-500"
          />
        </div>

        {/* Password */}
        <div>
          <label className="mb-1 block font-medium">
            Password
          </label>

          <input
            type="password"
            name="password"
            value={formData.password}
            onChange={handleChange}
            placeholder="Enter your password"
            className="w-full rounded-lg border px-4 py-2
                       outline-none focus:ring-2
                       focus:ring-indigo-500"
          />
        </div>

        <button
          type="submit"
          className="w-full rounded-lg bg-indigo-600
                     px-4 py-3 font-semibold text-white
                     transition hover:bg-indigo-700"
        >
          Register
        </button>

        {/* Message */}
        {message && (
          <p className="mt-4 text-center font-semibold text-indigo-600">
            {message}
          </p>
        )}

      </form>
    </div>
  </div>
);
```

```
}

export default Register;
```

TRY

Test Register

Go to:

/register

Try these cases.

Case 1 — Empty form

Case 2 — Short password

Case 3 — Valid form

5/10/2026

STATUS so far

```
event-management-system/
│
├── client/
│   ├── src/
│   │   ├── assets/
│   │   │
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── EventCard.jsx
│   │   │   ├── Button.jsx
│   │   │   ├── Modal.jsx
│   │   │   └── Loading.jsx
│   │   │
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   │
│   │   ├── hooks/
│   │   │   └── useAuth.js
│   │   │
│   │   ├── pages/
│   │   │   ├── Home.jsx
│   │   │   ├── Events.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── MyRegistrations.jsx
│   │   │   └── AdminDashboard.jsx
│   │   │
│   │   ├── services/
│   │   │   └── api.js
│   │   │
│   │   ├── utils/
│   │   │
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
```

```
|   |
|   └── package.json
|
└── server/
    ├── config/
    |   └── db.js
    ├── controllers/
    |   ├── authController.js
    |   └── eventController.js
    ├── middleware/
    |   ├── authMiddleware.js
    |   └── errorMiddleware.js
    ├── models/
    |   ├── User.js
    |   ├── Event.js
    |   └── Registration.js
    ├── routes/
    |   ├── authRoutes.js
    |   ├── eventRoutes.js
    |   └── registrationRoutes.js
    ├── utils/
    |   └── generateToken.js
    ├── .env
    ├── .gitignore
    ├── package.json
    └── server.js
```

client/src/App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Navbar from "./components/Navbar";

import Home from "./pages/Home";
import Events from "./pages/Events";
import Login from "./pages/Login";
import Register from "./pages/Register";
import MyRegistrations from "./pages/MyRegistrations";
import AdminDashboard from "./pages/AdminDashboard";

function App() {
  return (
    <BrowserRouter>
      <Navbar />

      <Routes>
        <Route path="/" element={<Home />} />

        <Route path="/events" element={<Events />} />

        <Route path="/login" element={<Login />} />

        <Route path="/register" element={<Register />} />

        <Route
          path="/my-registrations"
          element={<MyRegistrations />}
        />

        <Route
          path="/admin"
          element={<AdminDashboard />}
        />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

---

client/src/components/Navbar.jsx

```
import { Link } from "react-router-dom";
```

```
function Navbar() {
  return (
    <nav className="bg-slate-900 text-white">
      <div className="mx-auto flex max-w-7xl items-center justify-between
px-6 py-4">

        <h1 className="text-2xl font-bold">
          EventHub
        </h1>

        <div className="flex gap-6">

          <Link
            to="/"
            className="transition hover:text-indigo-400"
          >
            Home
          </Link>

          <Link
            to="/events"
            className="transition hover:text-indigo-400"
          >
            Events
          </Link>

          <Link
            to="/login"
            className="transition hover:text-indigo-400"
          >
            Login
          </Link>

          <Link
            to="/register"
            className="transition hover:text-indigo-400"
          >
            Register
          </Link>

        </div>

      </div>
    </nav>
  );
}

export default Navbar;
```

---

client/src/pages/Home.jsx

```
function Home() {
  return (
    <div className="min-h-screen">

      {/* Hero Section */}
      <section className="bg-slate-100 px-6 py-20 text-center">

        <h1 className="text-4xl font-bold text-slate-900 md:text-6xl">
          Discover Amazing Events
        </h1>

        <p className="mx-auto mt-4 max-w-2xl text-lg text-slate-600">
          Find exciting events, connect with people, and
          register for experiences that interest you.
        </p>

        <button
          className="mt-8 rounded-lg bg-indigo-600 px-6 py-3
                     font-semibold text-white transition
                     hover:bg-indigo-700"
        >
          Explore Events
        </button>

      </section>

      {/* Introduction Section */}
      <section className="bg-white px-6 py-16 text-center">

        <h2 className="text-3xl font-bold text-slate-900">
          Find Your Next Experience
        </h2>

        <p className="mx-auto mt-4 max-w-2xl text-slate-600">
          Explore technology workshops, conferences,
          festivals, and other exciting events happening
          near you.
        </p>

      </section>

    </div>
  );
}
```

```
export default Home;
```

---

client/src/components/EventCard.jsx

```
import { useState } from "react";

function EventCard({
  title,
  description,
  date,
  location,
}) {

  const [registered, setRegistered] = useState(false);

  return (
    <div className="rounded-xl border bg-white p-6 shadow-md">

      <h2 className="text-xl font-bold text-slate-900">
        {title}
      </h2>

      <p className="mt-2 text-slate-600">
        {description}
      </p>

      <p className="mt-3 text-slate-600">
        Date: {date}
      </p>

      <p className="text-slate-600">
        Location: {location}
      </p>

      {registered ? (

        <p className="mt-4 font-semibold text-green-600">
          Successfully Registered!
        </p>

      ) : (

        <button
          onClick={() => setRegistered(true)}
          className="mt-4 rounded-lg bg-indigo-600 px-4 py-2
                     text-white transition
                     hover:bg-indigo-700"
```

```
        >
          Register
        </button>

      )}

    </div>
  );
}

export default EventCard;
```

---

client/src/pages/Events.jsx

```
import EventCard from "../components/EventCard";

const events = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "20 September 2026",
    location: "Mumbai",
  },

  {
    id: 2,
    title: "React Workshop",
    description: "Learn React from basics to advanced.",
    date: "25 September 2026",
    location: "Pune",
  },

  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest developments in AI.",
    date: "30 September 2026",
    location: "Bangalore",
  },
];

function Events() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">
```

```
        <h1 className="text-4xl font-bold text-slate-900">
          Upcoming Events
        </h1>

        <p className="mt-2 text-slate-600">
          Explore our upcoming events and register
          for the ones you like.
        </p>

        <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2
lg:grid-cols-3">

          {events.map((event) => (

            <EventCard
              key={event.id}
              title={event.title}
              description={event.description}
              date={event.date}
              location={event.location}
            />

          ))}

        </div>

      </div>

    </div>
  );
}

export default Events;
```

---

client/src/pages/Register.jsx

```
import { useState } from "react";

function Register() {

  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
  });
```

```
  const [message, setMessage] = useState("");

  function handleChange(event) {

    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value,
    });
  }

  function handleSubmit(event) {

    event.preventDefault();

    if (
      !formData.name ||
      !formData.email ||
      !formData.password
    ) {
      setMessage("Please fill in all fields.");
      return;
    }

    if (formData.password.length < 6) {
      setMessage("Password must be at least 6 characters.");
      return;
    }

    setMessage("Registration successful!");
  }

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8
shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Create Account
        </h1>

        <p className="mt-2 text-slate-600">
          Register to start exploring events.
        </p>
```

```
<form
  onSubmit={handleSubmit}
  className="mt-6 space-y-4"
>

  {/* Name */}
  <div>
    <label className="mb-1 block font-medium">
      Name
    </label>

    <input
      type="text"
      name="name"
      value={formData.name}
      onChange={handleChange}
      placeholder="Enter your name"
      className="w-full rounded-lg border px-4 py-2
                 outline-none focus:ring-2
                 focus:ring-indigo-500"
    />
  </div>

  {/* Email */}
  <div>
    <label className="mb-1 block font-medium">
      Email
    </label>

    <input
      type="email"
      name="email"
      value={formData.email}
      onChange={handleChange}
      placeholder="Enter your email"
      className="w-full rounded-lg border px-4 py-2
                 outline-none focus:ring-2
                 focus:ring-indigo-500"
    />
  </div>

  {/* Password */}
  <div>
    <label className="mb-1 block font-medium">
      Password
    </label>

    <input
```

```
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
              placeholder="Enter your password"
              className="w-full rounded-lg border px-4 py-2
                         outline-none focus:ring-2
                         focus:ring-indigo-500"
            />
          </div>

          <button
            type="submit"
            className="w-full rounded-lg bg-indigo-600
                       px-4 py-3 font-semibold text-white
                       transition hover:bg-indigo-700"
          >
            Register
          </button>

          {message && (
            <p className="mt-4 text-center font-semibold
text-indigo-600">
              {message}
            </p>
          )}

        </form>

      </div>

    </div>
  );
}

export default Register;
```

---

client/src/pages/Login.jsx

```
import { useState } from "react";

function Login() {

  const [formData, setFormData] = useState({
    email: "",
    password: "",
```

```
  });

  const [message, setMessage] = useState("");

  function handleChange(event) {

    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value,
    });
  }

  function handleSubmit(event) {

    event.preventDefault();

    if (!formData.email || !formData.password) {
      setMessage("Please fill in all fields.");
      return;
    }

    setMessage("Login successful!");
  }

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8
shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Login
        </h1>

        <p className="mt-2 text-slate-600">
          Login to manage your event registrations.
        </p>

        <form
          onSubmit={handleSubmit}
          className="mt-6 space-y-4"
        >

          <div>
            <label className="mb-1 block font-medium">
              Email
```

```
      </label>

      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Enter your email"
        className="w-full rounded-lg border px-4 py-2
                   outline-none focus:ring-2
                   focus:ring-indigo-500"
      />
    </div>

    <div>
      <label className="mb-1 block font-medium">
        Password
      </label>

      <input
        type="password"
        name="password"
        value={formData.password}
        onChange={handleChange}
        placeholder="Enter your password"
        className="w-full rounded-lg border px-4 py-2
                   outline-none focus:ring-2
                   focus:ring-indigo-500"
      />
    </div>

    <button
      type="submit"
      className="w-full rounded-lg bg-indigo-600
                 px-4 py-3 font-semibold text-white
                 transition hover:bg-indigo-700"
    >
      Login
    </button>

    {message && (
      <p className="text-center font-semibold text-indigo-600">
        {message}
      </p>
    )}

  </form>
```

```
      </div>

    </div>
  );
}

export default Login;
```

MyRegistrations.jsx

```
function MyRegistrations() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          My Registrations
        </h1>

        <p className="mt-2 text-slate-600">
          Your registered events will appear here.
        </p>

      </div>

    </div>
  );
}

export default MyRegistrations;
```

AdminDashboard.jsx

```
function AdminDashboard() {
  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          Admin Dashboard
        </h1>

        <p className="mt-2 text-slate-600">
          Manage events and view registrations.
```

```
        </p>

      </div>

    </div>
  );
}

export default AdminDashboard;
```

RUN

```
cd client
npm install react-router-dom
npm run dev
```

Responsive Navbar

```
hidden md:flex
Mobile → hidden
Tablet/Desktop → flex
And:
md:hidden
means:
Mobile → visible
Tablet/Desktop → hidden
```

Replace `Navbar.jsx`
client/src/components/Navbar.jsx

```
import { useState } from "react";
import { Link } from "react-router-dom";

function Navbar() {
  const [menuOpen, setMenuOpen] = useState(false);

  return (
    <nav className="bg-slate-900 text-white">
      <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

        {/* Logo */}
        <Link to="/" className="text-2xl font-bold">
          EventHub
        </Link>
```

```
        {/* Desktop Menu */}
        <div className="hidden items-center gap-6 md:flex">

          <Link
            to="/"
            className="transition hover:text-indigo-400"
          >
            Home
          </Link>

          <Link
            to="/events"
            className="transition hover:text-indigo-400"
          >
            Events
          </Link>

          <Link
            to="/login"
            className="transition hover:text-indigo-400"
          >
            Login
          </Link>

          <Link
            to="/register"
            className="rounded-lg bg-indigo-600 px-4 py-2 font-semibold transition
hover:bg-indigo-700"
          >
            Register
          </Link>

        </div>

        {/* Mobile Menu Button */}
        <button
          onClick={() => setMenuOpen(!menuOpen)}
          className="rounded-lg border border-slate-600 px-3 py-2 md:hidden"
        >
          {menuOpen ? "✕" : "☰"}
        </button>

      </div>

      {/* Mobile Menu */}
      {menuOpen && (
        <div className="border-t border-slate-700 px-6 py-4 md:hidden">

          <div className="flex flex-col gap-4">

            <Link
              to="/"
              onClick={() => setMenuOpen(false)}
              className="transition hover:text-indigo-400"
            >
```

```
            Home
          </Link>

          <Link
            to="/events"
            onClick={() => setMenuOpen(false)}
            className="transition hover:text-indigo-400"
          >
            Events
          </Link>

          <Link
            to="/login"
            onClick={() => setMenuOpen(false)}
            className="transition hover:text-indigo-400"
          >
            Login
          </Link>

          <Link
            to="/register"
            onClick={() => setMenuOpen(false)}
            className="transition hover:text-indigo-400"
          >
            Register
          </Link>

        </div>

      </div>
    )}

  </nav>
 );
}

export default Navbar;
```

---

useEffect()

Syntax

```
useEffect(() => {
  // side effect
}, []);
```

two important parts:
Effect function
```
() => {
   // work
```

```
}
```

Dependency array
[]
The empty array means:
Run the effect when the component is mounted

client/src/components/Loading.jsx

```
function Loading() {
  return (
    <div className="flex min-h-40 items-center justify-center">
      <div className="flex items-center gap-3">

        <div className="h-6 w-6 animate-spin rounded-full border-4 border-slate-300
border-t-indigo-600"></div>

        <p className="font-medium text-slate-600">
          Loading events...
        </p>

      </div>
    </div>
  );
}

export default Loading;
```

`Events.jsx`
Replace the entire file:
client/src/pages/Events.jsx

```
import { useEffect, useState } from "react";

import EventCard from "../components/EventCard";
import Loading from "../components/Loading";

const eventData = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "20 September 2026",
    location: "Mumbai",
  },
  {
    id: 2,
    title: "React Workshop",
```

```
    description: "Learn React from basics to advanced.",
    date: "25 September 2026",
    location: "Pune",
  },
  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest developments in AI.",
    date: "30 September 2026",
    location: "Bangalore",
  },
];

function Events() {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    console.log("Events page loaded");

    const timer = setTimeout(() => {
      setEvents(eventData);
      setLoading(false);
    }, 1000);

    return () => {
      clearTimeout(timer);
      console.log("Events page cleanup");
    };
  }, []);

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          Upcoming Events
        </h1>

        <p className="mt-2 text-slate-600">
          Explore our upcoming events and register
          for the ones you like.
        </p>

        {loading ? (
          <Loading />
        ) : (
          <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
```

```
          {events.map((event) => (
            <EventCard
              key={event.id}
              title={event.title}
              description={event.description}
              date={event.date}
              location={event.location}
            />
          ))}

        </div>
      )}

    </div>

  </div>
  );
}

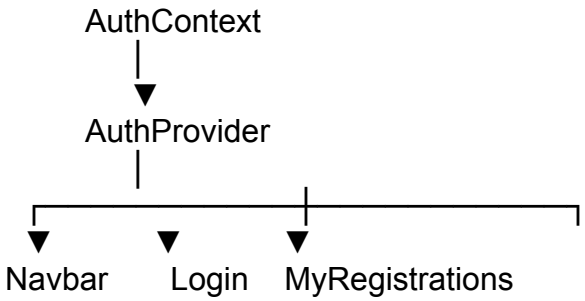
export default Events;
```

---

useContext()

Context API allows us to share data with many components without manually passing props through every level

Context architecture



AuthContext.jsx
client/src/context/AuthContext.jsx

```
import { createContext, useEffect, useState } from "react";

export const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
```

```
  useEffect(() => {
    const savedUser = localStorage.getItem("eventHubUser");

    if (savedUser) {
      setUser(JSON.parse(savedUser));
    }
  }, []);

  function login(email) {
    const loggedInUser = {
      name: email.split("@")[0],
      email: email,
    };

    setUser(loggedInUser);

    localStorage.setItem(
      "eventHubUser",
      JSON.stringify(loggedInUser)
    );
  }

  function logout() {
    setUser(null);

    localStorage.removeItem("eventHubUser");
  }

  return (
    <AuthContext.Provider
      value={{
        user,
        login,
        logout,
      }}
    >
      {children}
    </AuthContext.Provider>
  );
}

export default AuthProvider;
```

`main.jsx`

For the Provider to work, we must wrap the application.
client/src/main.jsx

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";

import App from "./App";
import AuthProvider from "./context/AuthContext";

import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </React.StrictMode>
);
```

Now the structure is:
main.jsx
↓
AuthProvider
↓
App
↓
Navbar
Login
Events
MyRegistrations
AdminDashboard
All of these can access the Context.

Open:
client/src/components/Navbar.jsx
At the top:

```
import { useContext, useState } from "react";
import { Link } from "react-router-dom";

import { AuthContext } from "../context/AuthContext";
```

Inside the component:

```
const { user, logout } = useContext(AuthContext);
```

Navbar.jsx

```
import { useContext, useState } from "react";
import { Link } from "react-router-dom";

import { AuthContext } from "../context/AuthContext";
```

```
function Navbar() {
  const [menuOpen, setMenuOpen] = useState(false);

  const { user, logout } = useContext(AuthContext);

  function closeMenu() {
    setMenuOpen(false);
  }

  function handleLogout() {
    logout();
    closeMenu();
  }

  return (
    <nav className="bg-slate-900 text-white">

      <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

        {/* Logo */}
        <Link
          to="/"
          onClick={closeMenu}
          className="text-2xl font-bold"
        >
          EventHub
        </Link>

        {/* Desktop Menu */}
        <div className="hidden items-center gap-6 md:flex">

          <Link
            to="/"
            className="transition hover:text-indigo-400"
          >
            Home
          </Link>

          <Link
            to="/events"
            className="transition hover:text-indigo-400"
          >
            Events
          </Link>

          {user ? (
            <>
              <Link
                to="/my-registrations"
                className="transition hover:text-indigo-400"
```

```
            >
              My Registrations
            </Link>

            <span className="text-sm text-slate-300">
              Hi, {user.name}
            </span>

            <button
              onClick={logout}
              className="rounded-lg bg-red-600 px-4 py-2 transition hover:bg-red-700"
            >
              Logout
            </button>
          </>
        ) : (
          <>
            <Link
              to="/login"
              className="transition hover:text-indigo-400"
            >
              Login
            </Link>

            <Link
              to="/register"
              className="rounded-lg bg-indigo-600 px-4 py-2 font-semibold transition
hover:bg-indigo-700"
            >
              Register
            </Link>
          </>
        )}

      </div>

      {/* Mobile Menu Button */}
      <button
        onClick={() => setMenuOpen(!menuOpen)}
        className="rounded-lg border border-slate-600 px-3 py-2 md:hidden"
      >
        {menuOpen ? "✕" : "☰"}
      </button>

    </div>

    {/* Mobile Menu */}
    {menuOpen && (
      <div className="border-t border-slate-700 px-6 py-4 md:hidden">
```

```
<div className="flex flex-col gap-4">

  <Link
    to="/"
    onClick={closeMenu}
  >
    Home
  </Link>

  <Link
    to="/events"
    onClick={closeMenu}
  >
    Events
  </Link>

  {user ? (
    <>
      <Link
        to="/my-registrations"
        onClick={closeMenu}
      >
        My Registrations
      </Link>

      <p className="text-sm text-slate-300">
        Logged in as {user.name}
      </p>

      <button
        onClick={handleLogout}
        className="rounded-lg bg-red-600 px-4 py-2 text-left"
      >
        Logout
      </button>
    </>
  ) : (
    <>
      <Link
        to="/login"
        onClick={closeMenu}
      >
        Login
      </Link>

      <Link
        to="/register"
        onClick={closeMenu}
      >
        Register
```

```
            </Link>
          </>
        )}

      </div>

    </div>
  )}

 </nav>
 );
}

export default Navbar;
```

Update Login
Now demonstrate that another component can access the same Context.
File:
client/src/pages/Login.jsx

```
import { useContext, useState } from "react";
import { useNavigate } from "react-router-dom";

import { AuthContext } from "../context/AuthContext";

function Login() {
 const { login } = useContext(AuthContext);

 const navigate = useNavigate();

 const [formData, setFormData] = useState({
  email: "",
  password: "",
 });

 const [message, setMessage] = useState("");

 function handleChange(event) {
  const { name, value } = event.target;

  setFormData({
   ...formData,
   [name]: value,
  });
 }

 function handleSubmit(event) {
  event.preventDefault();
```

```
    if (!formData.email || !formData.password) {
      setMessage("Please fill in all fields.");
      return;
    }

    login(formData.email);

    setMessage("Login successful!");

    setTimeout(() => {
      navigate("/");
    }, 500);
  }

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Login
        </h1>

        <p className="mt-2 text-slate-600">
          Login to manage your event registrations.
        </p>

        <form
          onSubmit={handleSubmit}
          className="mt-6 space-y-4"
        >

          <div>
            <label className="mb-1 block font-medium">
              Email
            </label>

            <input
              type="email"
              name="email"
              value={formData.email}
              onChange={handleChange}
              placeholder="Enter your email"
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
            />
          </div>

          <div>
```

```
            <label className="mb-1 block font-medium">
              Password
            </label>

            <input
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
              placeholder="Enter your password"
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
            />
          </div>

          <button
            type="submit"
            className="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white
transition hover:bg-indigo-700"
          >
            Login
          </button>

          {message && (
            <p className="text-center font-semibold text-green-600">
              {message}
            </p>
          )}

        </form>

      </div>

    </div>
  );
}

export default Login;
```

Login and check

Custom Hooks

useAuth()
Instead of writing:
useContext(AuthContext)
everywhere, we will write:
useAuth()

Create `useAuth.js`
File:
client/src/hooks/useAuth.js

```
import { useContext } from "react";

import { AuthContext } from "../context/AuthContext";

function useAuth() {
  return useContext(AuthContext);
}

export default useAuth;
```

Replace Navbar's `useContext()`
Remove:
`import { useContext, useState } from "react";`
and use:
`import { useState } from "react";`
Remove:
`import { AuthContext } from "../context/AuthContext";`
Add:
`import useAuth from "../hooks/useAuth";`
Then change:
`const { user, logout } = useContext(AuthContext);`
to:
`const { user, logout } = useAuth();`

`Navbar.jsx`

```
import { useState } from "react";
import { Link } from "react-router-dom";

import useAuth from "../hooks/useAuth";

function Navbar() {
  const [menuOpen, setMenuOpen] = useState(false);

  const { user, logout } = useAuth();

  function closeMenu() {
    setMenuOpen(false);
  }

  function handleLogout() {
    logout();
    closeMenu();
```

```
}

return (
  <nav className="bg-slate-900 text-white">

    <div className="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

      {/* Logo */}
      <Link
        to="/"
        onClick={closeMenu}
        className="text-2xl font-bold"
      >
        EventHub
      </Link>

      {/* Desktop Menu */}
      <div className="hidden items-center gap-6 md:flex">

        <Link
          to="/"
          className="transition hover:text-indigo-400"
        >
          Home
        </Link>

        <Link
          to="/events"
          className="transition hover:text-indigo-400"
        >
          Events
        </Link>

        {user ? (
          <>
            <Link
              to="/my-registrations"
              className="transition hover:text-indigo-400"
            >
              My Registrations
            </Link>

            <span className="text-sm text-slate-300">
              Hi, {user.name}
            </span>

            <button
              onClick={logout}
              className="rounded-lg bg-red-600 px-4 py-2 transition hover:bg-red-700"
            >
```

```
            Logout
          </button>
        </>
      ) : (
        <>
          <Link
            to="/login"
            className="transition hover:text-indigo-400"
          >
            Login
          </Link>

          <Link
            to="/register"
            className="rounded-lg bg-indigo-600 px-4 py-2 font-semibold transition
hover:bg-indigo-700"
          >
            Register
          </Link>
        </>
      )}

    </div>

    {/* Mobile Menu Button */}
    <button
      onClick={() => setMenuOpen(!menuOpen)}
      className="rounded-lg border border-slate-600 px-3 py-2 md:hidden"
    >
      {menuOpen ? "✕" : "☰"}
    </button>

  </div>

  {/* Mobile Menu */}
  {menuOpen && (
    <div className="border-t border-slate-700 px-6 py-4 md:hidden">

      <div className="flex flex-col gap-4">

        <Link
          to="/"
          onClick={closeMenu}
        >
          Home
        </Link>

        <Link
          to="/events"
          onClick={closeMenu}
```

```
            >
              Events
            </Link>

            {user ? (
              <>
                <Link
                  to="/my-registrations"
                  onClick={closeMenu}
                >
                  My Registrations
                </Link>

                <p className="text-sm text-slate-300">
                  Logged in as {user.name}
                </p>

                <button
                  onClick={handleLogout}
                  className="rounded-lg bg-red-600 px-4 py-2 text-left"
                >
                  Logout
                </button>
              </>
            ) : (
              <>
                <Link
                  to="/login"
                  onClick={closeMenu}
                >
                  Login
                </Link>

                <Link
                  to="/register"
                  onClick={closeMenu}
                >
                  Register
                </Link>
              </>
            )}

          </div>

        </div>
      )}

    </nav>
  );
}
```

```
export default Navbar;
```

Replace Login's `useContext()`
Remove:
```
import { useContext, useState } from "react";
```
Replace with:
```
import { useState } from "react";
```
Remove:
```
import { AuthContext } from "../context/AuthContext";
```
Add:
```
import useAuth from "../hooks/useAuth";
```
Then:
```
const { login } = useAuth();
```

`Login.jsx`

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

import useAuth from "../hooks/useAuth";

function Login() {
  const { login } = useAuth();

  const navigate = useNavigate();

  const [formData, setFormData] = useState({
    email: "",
    password: "",
  });

  const [message, setMessage] = useState("");

  function handleChange(event) {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value,
    });
  }

  function handleSubmit(event) {
    event.preventDefault();

    if (!formData.email || !formData.password) {
```

```
      setMessage("Please fill in all fields.");
      return;
    }

    login(formData.email);

    setMessage("Login successful!");

    setTimeout(() => {
      navigate("/");
    }, 500);
  }

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">

        <h1 className="text-3xl font-bold text-slate-900">
          Login
        </h1>

        <p className="mt-2 text-slate-600">
          Login to manage your event registrations.
        </p>

        <form
          onSubmit={handleSubmit}
          className="mt-6 space-y-4"
        >

          <div>
            <label className="mb-1 block font-medium">
              Email
            </label>

            <input
              type="email"
              name="email"
              value={formData.email}
              onChange={handleChange}
              placeholder="Enter your email"
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
            />
          </div>

          <div>
            <label className="mb-1 block font-medium">
              Password
```

```
          </label>

          <input
            type="password"
            name="password"
            value={formData.password}
            onChange={handleChange}
            placeholder="Enter your password"
            className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
          />
        </div>

        <button
          type="submit"
          className="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white
transition hover:bg-indigo-700"
        >
          Login
        </button>

        {message && (
          <p className="text-center font-semibold text-green-600">
            {message}
          </p>
        )}

      </form>

    </div>

  </div>
 );
}

export default Login;
```

Reuse the Custom Hook in MyRegistrations

File:
client/src/pages/MyRegistrations.jsx

```
import useAuth from "../hooks/useAuth";

function MyRegistrations() {
 const { user } = useAuth();

 return (
```

```
    <div className="min-h-screen bg-slate-100 px-6 py-16">

      <div className="mx-auto max-w-7xl">

        <h1 className="text-4xl font-bold text-slate-900">
          My Registrations
        </h1>

        {user ? (
          <p className="mt-4 text-slate-600">
            Welcome,{" "}
            <span className="font-semibold">
              {user.name}
            </span>
            . Your registered events will appear here.
          </p>
        ) : (
          <p className="mt-4 text-red-600">
            Please login to view your registrations.
          </p>
        )}

      </div>

    </div>
  );
}

export default MyRegistrations;
```

Summary

Navbar.jsx
↓
useAuth()

Login.jsx
↓
useAuth()

MyRegistrations.jsx
↓
useAuth()

All three are accessing:
AuthContext
through the same custom hook.

12/10/2026

Backend / Express Server and Test the REST API in Postman

GET /api/events
GET /api/events/:id

POST /api/events

PUT /api/events/:id

DELETE /api/events/:id

We shall create

```
server/
├── controllers/
│   ├── authController.js
│   └── eventController.js
│
├── routes/
│   ├── authRoutes.js
│   ├── eventRoutes.js
│   └── registrationRoutes.js
│
└── server.js
```

Client
↓
Route
↓
Controller
↓
Database
↓
Response

Start the Express backend

cd server
npm install express
npm install --save-dev nodemon

Configure `server/package.json`

do not replace the entire file. Add or update the `scripts` section, keeping your existing dependencies.

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Configure server/server.js

```
const express = require("express");
const app = express();

const eventRoutes = require("./routes/eventRoutes");

// Middleware to read JSON request bodies
app.use(express.json());

// Home route
app.get("/", (req, res) => {
  res.status(200).send("Event Management API Running");
});

// Event API routes
app.use("/api/events", eventRoutes);

// Handle unknown routes
app.use((req, res) => {
  res.status(404).json({
    message: "Route not found",
  });
});

// Start server
const PORT = 5000;
```

```
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Run the server
```
npm run dev
```

Open `http://localhost:5000` in your browser.

Create the event controller

Open `server/controllers/`**`eventController.js`**

```
let events = [
  {
    id: 1,
    title: "Tech Fest 2026",
    description: "A technology event for students.",
    date: "2026-11-15",
    time: "10:00",
    location: "Mumbai",
  },
  {
    id: 2,
    title: "React Workshop",
    description: "Learn React from basics to advanced.",
    date: "2026-11-20",
    time: "14:00",
    location: "Pune",
  },
  {
    id: 3,
    title: "AI Conference",
    description: "Explore the latest AI developments.",
    date: "2026-11-25",
    time: "09:30",
    location: "Bengaluru",
  },
];

// GET /api/events
function getEvents(req, res) {
  res.status(200).json(events);
}

// GET /api/events/:id
```

```
function getEventById(req, res) {
  const id = Number(req.params.id);

  const event = events.find((event) => event.id === id);

  if (!event) {
    return res.status(404).json({
      message: "Event not found",
    });
  }

  res.status(200).json(event);
}

// POST /api/events
function createEvent(req, res) {
  const { title, description, date, time, location } = req.body;

  if (
    !title?.trim() ||
    !description?.trim() ||
    !date?.trim() ||
    !time?.trim() ||
    !location?.trim()
  ) {
    return res.status(400).json({
      message: "All event fields are required",
    });
  }

  const newEvent = {
    id: events.length
      ? Math.max(...events.map((event) => event.id)) + 1
      : 1,
    title: title.trim(),
    description: description.trim(),
    date: date.trim(),
    time: time.trim(),
    location: location.trim(),
  };

  events.push(newEvent);

  res.status(201).json({
    message: "Event created successfully",
    event: newEvent,
  });
}
```

```
// PUT /api/events/:id
function updateEvent(req, res) {
  const id = Number(req.params.id);

  const index = events.findIndex((event) => event.id === id);

  if (index === -1) {
    return res.status(404).json({
      message: "Event not found",
    });
  }

  const { title, description, date, time, location } = req.body;

  if (
    !title?.trim() ||
    !description?.trim() ||
    !date?.trim() ||
    !time?.trim() ||
    !location?.trim()
  ) {
    return res.status(400).json({
      message: "All event fields are required",
    });
  }

  events[index] = {
    id,
    title: title.trim(),
    description: description.trim(),
    date: date.trim(),
    time: time.trim(),
    location: location.trim(),
  };

  res.status(200).json({
    message: "Event updated successfully",
    event: events[index],
  });
}

// DELETE /api/events/:id
function deleteEvent(req, res) {
  const id = Number(req.params.id);

  const index = events.findIndex((event) => event.id === id);
```

```
  if (index === -1) {
    return res.status(404).json({
      message: "Event not found",
    });
  }

  const deletedEvent = events[index];

  events = events.filter((event) => event.id !== id);

  res.status(200).json({
    message: "Event deleted successfully",
    event: deletedEvent,
  });
}

module.exports = {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent,
};
```

Create the event routes

Open `server/routes/eventRoutes.js`

```
const express = require("express");

const router = express.Router();

const {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent,
} = require("../controllers/eventController");

// GET all events
router.get("/", getEvents);

// GET one event
router.get("/:id", getEventById);
```

```
// CREATE an event
router.post("/", createEvent);

// UPDATE an event
router.put("/:id", updateEvent);

// DELETE an event
router.delete("/:id", deleteEvent);

module.exports = router;
```

Restart and test the API

```
cd server
npm run dev
```

Test the first two endpoints directly in your browser:

- http://localhost:5000/
- http://localhost:5000/api/events

For POST, PUT, and DELETE, use Postman.

Postman testing guide

GET

1. Get all events

URL: http://localhost:5000/api/events

Expected: 200 OK and a JSON array of three events.

GET

2. Get one event

URL: http://localhost:5000/api/events/1

Expected: 200 OK and the Tech Fest event.

POST

3. Create an event

URL: http://localhost:5000/api/events

Select Body → raw → JSON, and send the example below.

PUT

4. Update an event

URL: http://localhost:5000/api/events/1

Send all five fields in JSON, including the changed title.

DELETE

5. Delete an event

URL: http://localhost:5000/api/events/1

Expected: 200 OK and a confirmation response.

Connect React to the backend

The key change is this:

- Before: `Events.jsx` uses a hard-coded array.
- After: `Events.jsx` requests event data from Express using `fetch()`.

Open `client/vite.config.js`

Add the `server.proxy` configuration to the existing `defineConfig` object:

```
server: {
  proxy: {
    "/api": "http://localhost:5000",
  },
},
```

Sample

```
import { defineConfig } from "vite";
```

```
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],

  server: {
    proxy: {
      "/api": "http://localhost:5000",
    },
  },
});
```

This proxy forwards frontend requests beginning with `/api` to Express. It is a Vite development-server setting, so restart the frontend after changing it.

Open `client/src/services/api.js`.

```
export async function getEvents() {
  const response = await fetch("/api/events");

  if (!response.ok) {
    throw new Error("Failed to fetch events");
  }

  return response.json();
}
```

This function sends a GET request and returns the JSON data.

Update `Events.jsx`
Replace the contents of `client/src/pages/Events.jsx` with the following code:

```
import { useEffect, useState } from "react";

import EventCard from "../components/EventCard";
import Loading from "../components/Loading";
import { getEvents } from "../services/api";

function Events() {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    async function loadEvents() {
```

```
    try {
      const data = await getEvents();
      setEvents(data);
    } catch (err) {
      setError(err.message || "Unable to load events");
    } finally {
      setLoading(false);
    }
  }

  loadEvents();
}, []);

return (
  <div className="min-h-screen bg-slate-100 px-6 py-16">
    <div className="mx-auto max-w-7xl">

      <h1 className="text-4xl font-bold text-slate-900">
        Upcoming Events
      </h1>

      <p className="mt-2 text-slate-600">
        Explore our upcoming events and register.
      </p>

      {loading && <Loading />}

      {!loading && error && (
        <p className="mt-6 text-red-600">
          {error}
        </p>
      )}

      {!loading && !error && events.length === 0 && (
        <p className="mt-6 text-slate-600">
          No events are available.
        </p>
      )}

      {!loading && !error && events.length > 0 && (
        <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
          {events.map((event) => (
            <EventCard
              key={event.id}
              title={event.title}
              description={event.description}
              date={event.date}
              location={event.location}
```

```
              time={event.time}
            />
          ))}
        </div>
      ))}

    </div>
  </div>
 );
}

export default Events;
```

Update `EventCard.jsx` to show the time

In `client/src/components/EventCard.jsx`, update the function declaration:

```
function EventCard({
  title,
  description,
  date,
  time,
  location,
}) {
```

Then add this paragraph after the date paragraph:

```
<p className="text-slate-600">
  Time: {time}
</p>
```

Run both applications

Terminal 1 — backend:

```
cd server
npm run dev
```

Terminal 2 — frontend:

```
cd client
npm run dev
```

Open the frontend URL printed by Vite, usually `http://localhost:5173`, then visit the Events page.

You should see the three events coming from the Express API rather than the hard-coded `eventData` array.

19/10/2026 - code so far

REST API Design with MongoDB + Mongoose

Technology - Purpose in EventHub

MongoDB- Stores event details in a database

Mongoose - Defines schemas and interacts with MongoDB

Express.js - Creates API endpoints

REST API - Defines how the frontend requests and modifies data

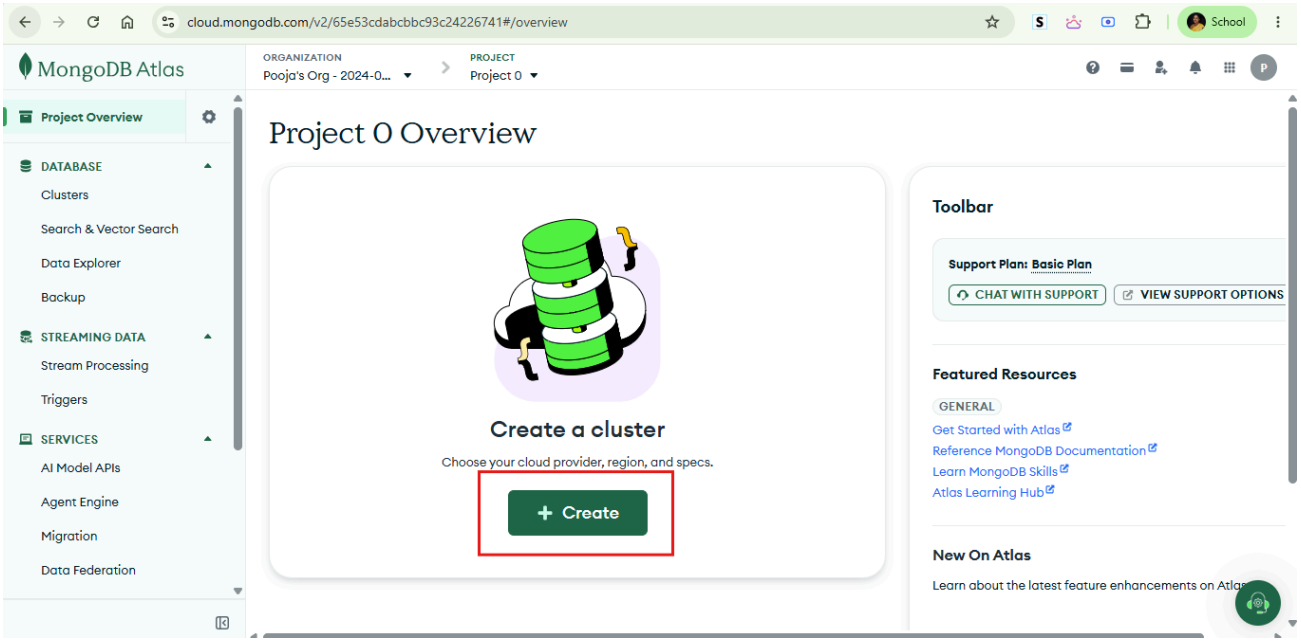
Controller- Contains the logic for each API operation

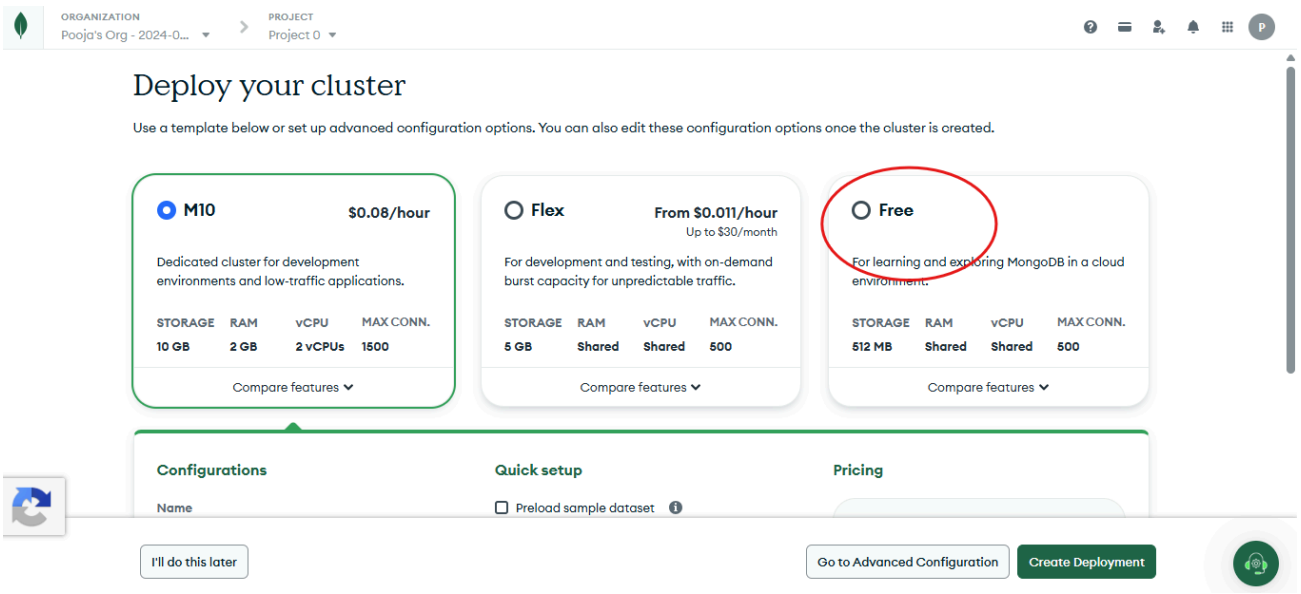
Create a free MongoDB Atlas cluster

Create a MongoDB Atlas cluster

https://www.mongodb.com/

- create a project named `EventHub` and continue.

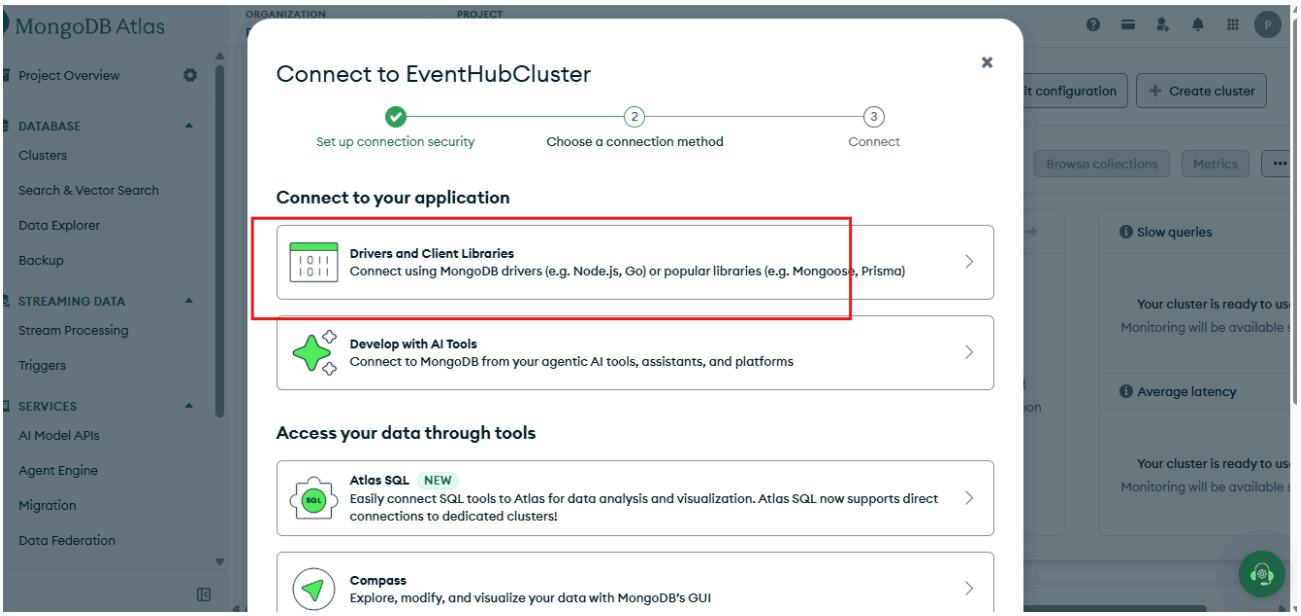




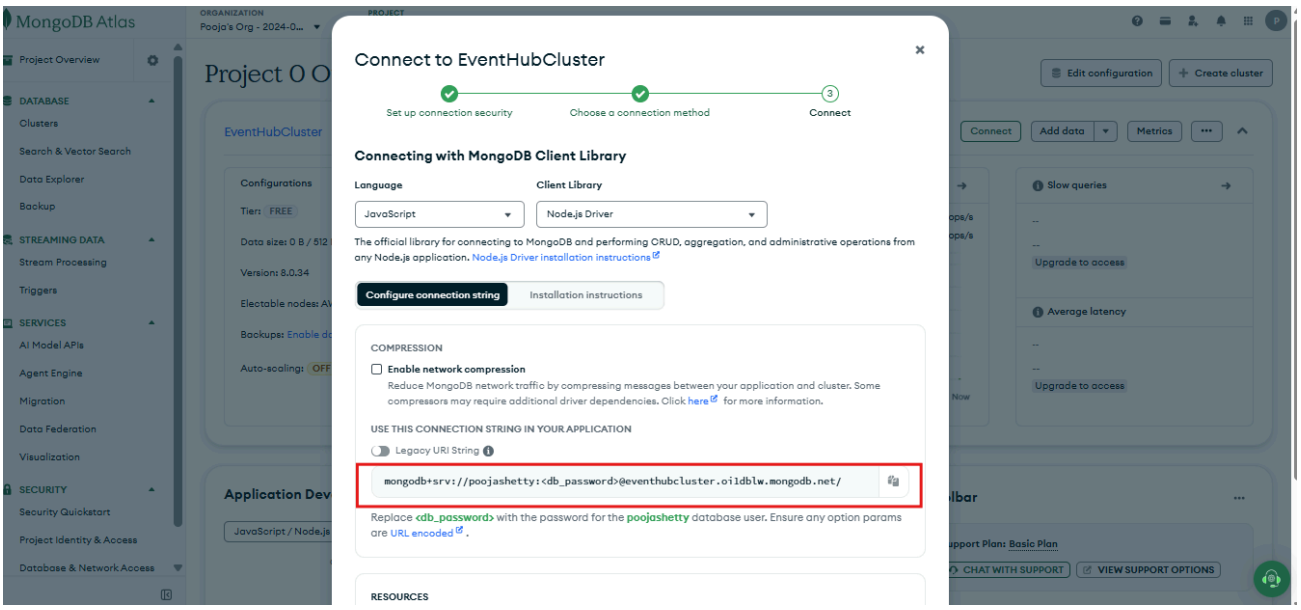
- Select M0 Free. Do not select a paid Flex or Dedicated option.
- Choose a cloud provider such as AWS.
- Select a region close to you, preferably Mumbai if a free-tier-supported region is available.
- In the cluster name field, enter:
  `EventHubCluster`

8. Click Create Deployment or Create.
9. Select Drivers and Client Libraries

    a. Click the first option: Drivers and Client Libraries



10. Choose Node.js
11. Find the Driver dropdown and select Node.js.
    a. Select the version recommended by Atlas.
    b. Atlas will display a connection string.

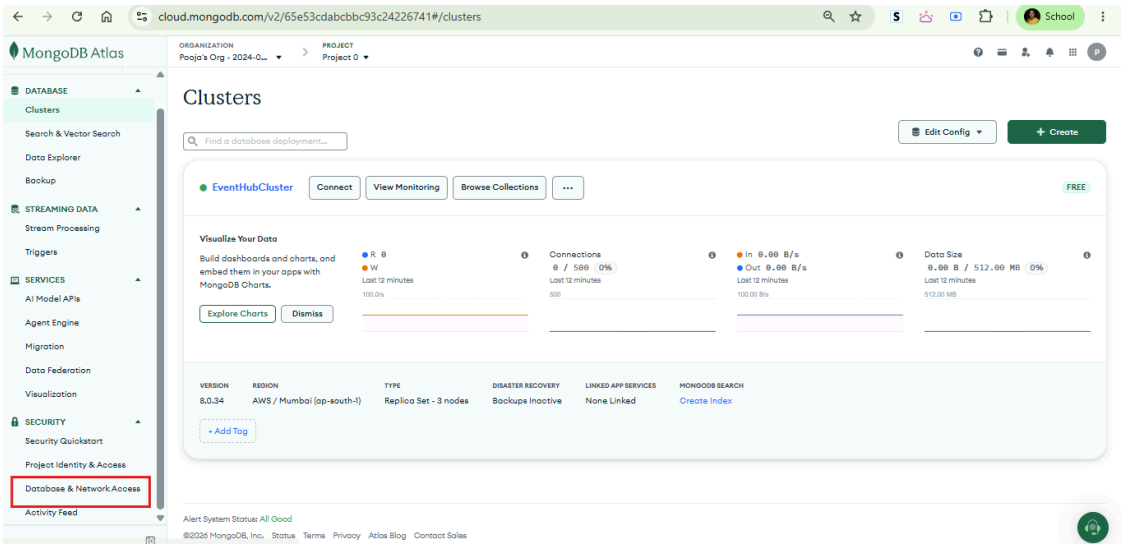


mongodb+srv://poojashetty:<db_password>@eventhubcluster.oi1dblw.mongodb.net/

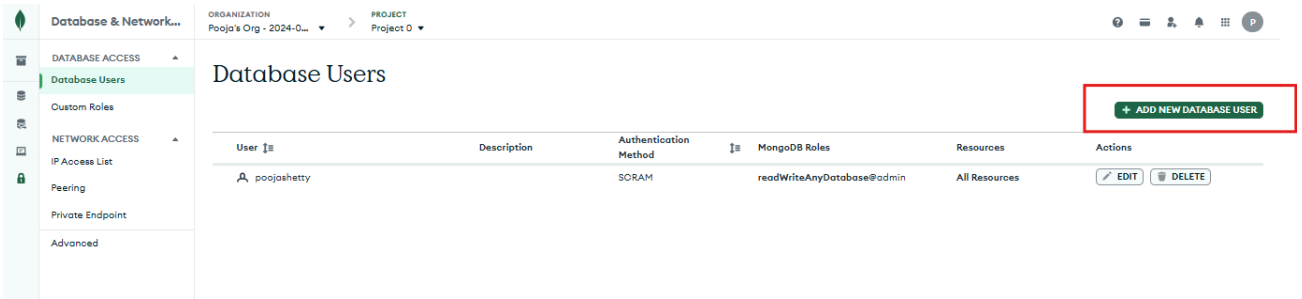
12. Check database access

Before connecting your application, make sure you've completed these two security settings:

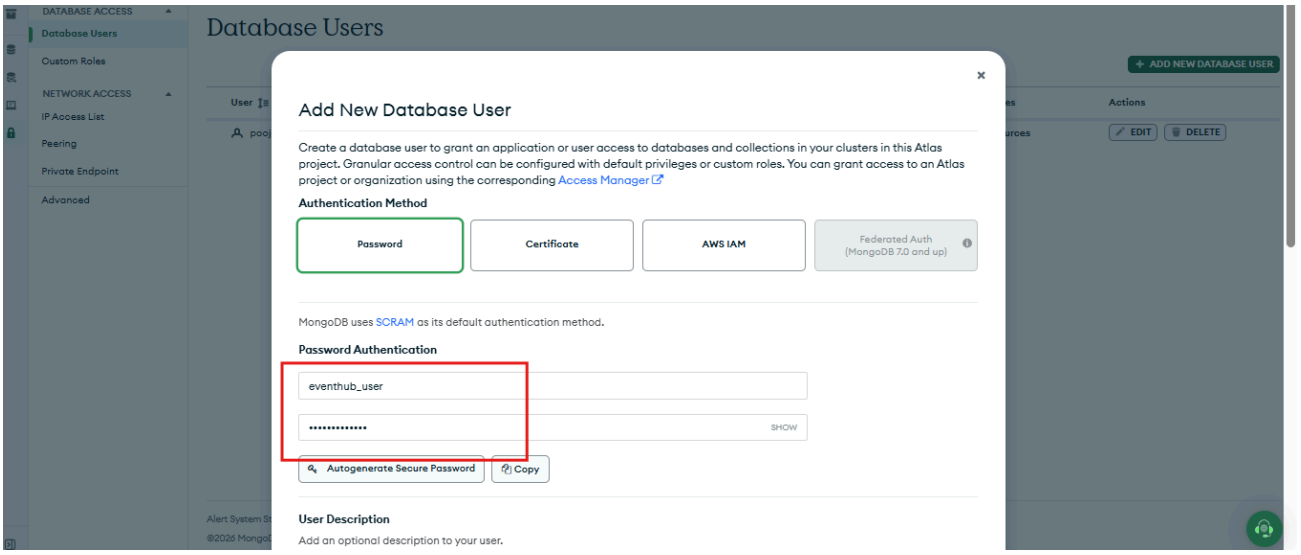
- Database Access: Created a database username and password.

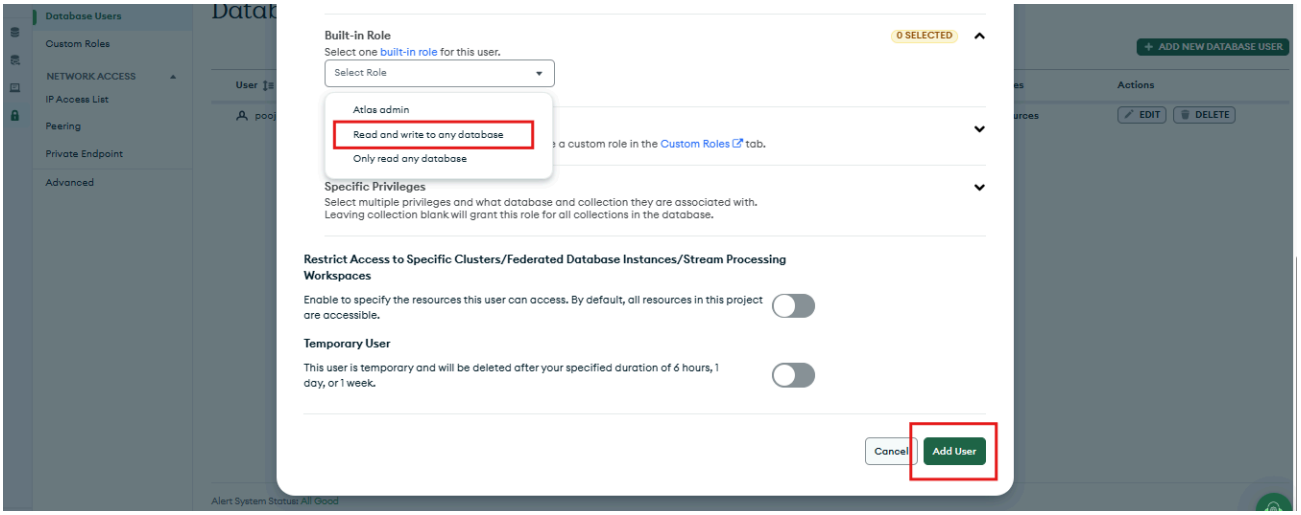


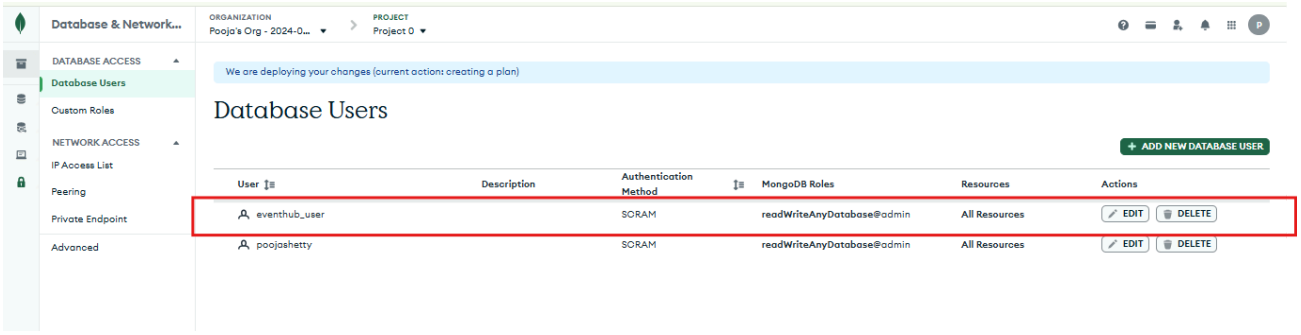
- Add New Database User or Add Database User.



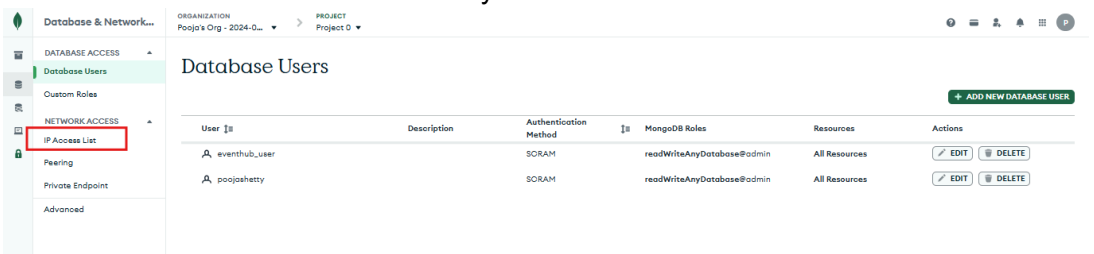
- Enter the username and password



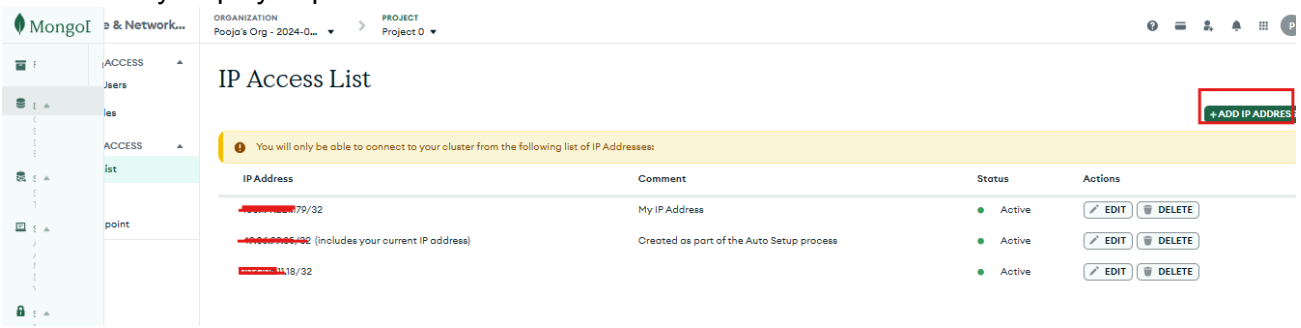




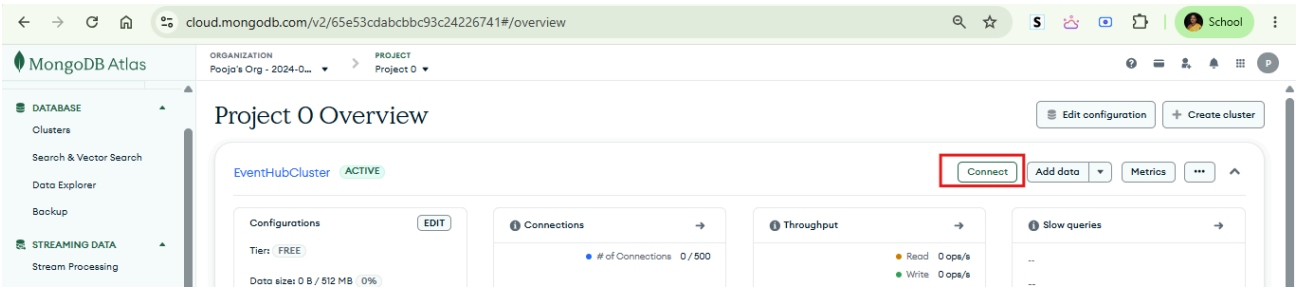
- Database & Network Access: Added your current IP address to the access list.

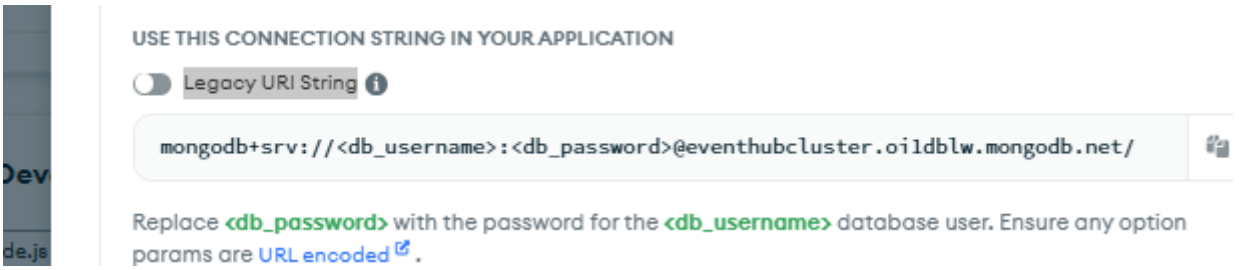


Use currently displayed public IP address or create one



- open your deployment and click Connect.





Configure server/.env

Open the .env file you already created inside the server folder.

PORT=5000
MONGODB_URI=mongodb+srv://YOUR_USERNAME:YOUR_PASSWORD@YOUR_CLUSTER_HOST/eventhub?retryWrites=true&w=majority

Replace the username, password, and cluster host with the values from your Atlas connection string.
Ensure `.env` is listed in `server/.gitignore`.

Connect Express to MongoDB

Open `server/config/db.js` and replace its contents with:
const mongoose = require("mongoose");

```
async function connectDB() {
  try {
    if (!process.env.MONGODB_URI) {
```

```
      throw new Error("MONGODB_URI is missing from .env");
    }

    await mongoose.connect(process.env.MONGODB_URI);

    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

`mongoose.connect()` establishes the database connection. The `await` ensures the connection attempt completes before your server begins accepting requests.

Next, open `server/server.js` and replace its contents with:

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");

const connectDB = require("./config/db");
const eventRoutes = require("./routes/eventRoutes");

const app = express();

app.use(cors());
app.use(express.json());

// Home route
app.get("/", (req, res) => {
  res.status(200).send("Event Management API Running");
});

// Existing REST API endpoints
app.use("/api/events", eventRoutes);

// Handle unknown routes
app.use((req, res) => {
  res.status(404).json({
    message: "Route not found",
  });
});

const PORT = process.env.PORT || 5000;
```

```
async function startServer() {
  await connectDB();

  app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
  });
}

startServer();
```

Your `eventRoutes.js` can remain unchanged because its five routes already point to your controller.

Create the Mongoose Event model

Open `server/models/Event.js`.

Replace its contents with:

```
const mongoose = require("mongoose");

const eventSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "Event title is required"],
      trim: true,
    },

    description: {
      type: String,
      required: [true, "Event description is required"],
      trim: true,
    },

    date: {
      type: Date,
      required: [true, "Event date is required"],
    },

    time: {
      type: String,
      required: [true, "Event time is required"],
      trim: true,
    },
```

```
    location: {
      type: String,
      required: [true, "Event location is required"],
      trim: true,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model("Event", eventSchema);
```

Field	Data type	Purpose
`title`	String	Event name
`description`	String	Event details
`date`	Date	Event date
`time`	String	Event time
`location`	String	Event venue
`createdAt`	Date	Automatically recorded creation time
`updatedAt`	Date	Automatically recorded modification time

Mongoose uses this schema to define the structure and validation rules for event documents.

Replace the in-memory event controller

Currently, your controller starts with something similar to:

```
let events = [
  // Three sample events
];
```

That array exists only in server memory. The data is lost when the process restarts, and it isn't shared with MongoDB.

Open `server/controllers/eventController.js` and replace the entire file with the code below.

```
const mongoose = require("mongoose");
```

```
const Event = require("../models/Event");

// GET /api/events
// Retrieve all events
async function getEvents(req, res) {
  try {
    const events = await Event.find().sort({ date: 1 });

    res.status(200).json(events);
  } catch (error) {
    console.error("Error retrieving events:", error.message);

    res.status(500).json({
      message: "Failed to retrieve events",
    });
  }
}

// GET /api/events/:id
// Retrieve one event
async function getEventById(req, res) {
  try {
    const { id } = req.params;

    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({
        message: "Invalid event ID",
      });
    }

    const event = await Event.findById(id);

    if (!event) {
      return res.status(404).json({
        message: "Event not found",
      });
    }

    res.status(200).json(event);
  } catch (error) {
    console.error("Error retrieving event:", error.message);

    res.status(500).json({
      message: "Failed to retrieve event",
    });
  }
}
```

```
// POST /api/events
// Create an event
async function createEvent(req, res) {
  try {
    const { title, description, date, time, location } = req.body;

    if (
      typeof title !== "string" ||
      !title.trim() ||
      typeof description !== "string" ||
      !description.trim() ||
      typeof date !== "string" ||
      !date.trim() ||
      typeof time !== "string" ||
      !time.trim() ||
      typeof location !== "string" ||
      !location.trim()
    ) {
      return res.status(400).json({
        message: "All event fields are required",
      });
    }

    const event = await Event.create({
      title: title.trim(),
      description: description.trim(),
      date,
      time: time.trim(),
      location: location.trim(),
    });

    res.status(201).json({
      message: "Event created successfully",
      event,
    });
  } catch (error) {
    console.error("Error creating event:", error.message);

    if (
      error.name === "ValidationError" ||
      error.name === "CastError"
    ) {
      return res.status(400).json({
        message: "Invalid event data",
        error: error.message,
      });
    }
```

```
    res.status(500).json({
      message: "Failed to create event",
    });
  }
}

// PUT /api/events/:id
// Update an event
async function updateEvent(req, res) {
  try {
    const { id } = req.params;

    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({
        message: "Invalid event ID",
      });
    }

    const { title, description, date, time, location } = req.body;

    if (
      typeof title !== "string" ||
      !title.trim() ||
      typeof description !== "string" ||
      !description.trim() ||
      typeof date !== "string" ||
      !date.trim() ||
      typeof time !== "string" ||
      !time.trim() ||
      typeof location !== "string" ||
      !location.trim()
    ) {
      return res.status(400).json({
        message: "All event fields are required",
      });
    }

    const event = await Event.findByIdAndUpdate(
      id,
      {
        title: title.trim(),
        description: description.trim(),
        date,
        time: time.trim(),
        location: location.trim(),
      },
      {
        new: true,
```

```
        runValidators: true,
      }
    );

    if (!event) {
      return res.status(404).json({
        message: "Event not found",
      });
    }

    res.status(200).json({
      message: "Event updated successfully",
      event,
    });
  } catch (error) {
    console.error("Error updating event:", error.message);

    if (
      error.name === "ValidationError" ||
      error.name === "CastError"
    ) {
      return res.status(400).json({
        message: "Invalid event data",
        error: error.message,
      });
    }

    res.status(500).json({
      message: "Failed to update event",
    });
  }
}

// DELETE /api/events/:id
// Delete an event
async function deleteEvent(req, res) {
  try {
    const { id } = req.params;

    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({
        message: "Invalid event ID",
      });
    }

    const event = await Event.findByIdAndDelete(id);

    if (!event) {
```

```
      return res.status(404).json({
        message: "Event not found",
      });
    }

    res.status(200).json({
      message: "Event deleted successfully",
      event,
    });
  } catch (error) {
    console.error("Error deleting event:", error.message);

    res.status(500).json({
      message: "Failed to delete event",
    });
  }
}

module.exports = {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent,
};
```

What changed in the controller?

Existing implementation	New implementation
`events.find(...)`	`Event.findById(...)`
`events.push(...)`	`Event.create(...)`
`events[index] = ...`	`Event.findByIdAndUpdate(...)`
`events.filter(...)`	`Event.findByIdAndDelete(...)`
Array stored in memory	Documents stored in MongoDB

The request URLs and HTTP methods remain the same. The IDs are different, though: MongoDB generates an `_id` for every event, rather than the numeric IDs `1`, `2`, and `3` in your original array.

Start the backend and verify the connection

Open a terminal in VS Code:

```
cd E:\event-management-system\server
```

npm run dev

Now open:

`http://localhost:5000/`

Expected response:

Event Management API Running

Then visit:

`http://localhost:5000/api/events`

If the database is new and has no event documents yet, the expected response is:

[]

That empty array is normal. Your previous three sample events were stored only in memory. They have not automatically been copied into MongoDB.

Create your first MongoDB event with Postman

Start Postman and make a `POST` request to:

http://localhost:5000/api/events

Select Body → raw → JSON and enter:

```
{
  "title": "Tech Fest 2026",
  "description": "A technology event for students.",
  "date": "2026-11-15",
  "time": "10:00",
  "location": "Mumbai"
}
```

Click Send.

Expected status: `201 Created`

The response should contain the created event and a generated ID, similar to:

```
{
  "message": "Event created successfully",
  "event": {
    "_id": "generated_mongodb_id",
    "title": "Tech Fest 2026",
    "description": "A technology event for students.",
    "date": "2026-11-15T00:00:00.000Z",
```

```
  "time": "10:00",
  "location": "Mumbai"
 }
}
```

The exact `_id` and timestamp will be generated by your application and database.

Return to `GET /api/events` and confirm that the new event is present. You can also check Atlas → Browse Collections.

Repeat using your existing Postman tests to verify all five endpoints:

Method	URL	Expected result
GET	`/api/events`	Returns saved events
GET	`/api/events/ACTUAL_MONGODB_ID`	Returns one event
POST	`/api/events`	Creates a document
PUT	`/api/events/ACTUAL_MONGODB_ID`	Updates a document
DELETE	`/api/events/ACTUAL_MONGODB_ID`	Deletes a document

Connect your React Events page to the MongoDB-backed API

0pen `client/src/pages/Events.jsx`.

Find this line:

```
key={event.id}
```

Replace it with:

```
key={event._id}
```

MongoDB returns `_id`, not the numeric `id` used by your old hard-coded array.

Your mapping should therefore look like this:

```
{events.map((event) => (
 <EventCard
  key={event._id}
  title={event.title}
  description={event.description}
  date={event.date}
  time={event.time}
```

```
    location={event.location}
  />
))}
```

In `client/src/components/EventCard.jsx`, check the code for

```
function EventCard({
  title,
  description,
  date,
  time,
  location,
}) {
  return (
    <div className="rounded-xl border bg-white p-6 shadow-md">
      <h2 className="text-xl font-bold text-slate-900">
        {title}
      </h2>

      <p className="mt-2 text-slate-600">
        {description}
      </p>

      <p className="mt-3 text-slate-600">
        Date:{" "}
        {new Date(date).toLocaleDateString("en-IN", {
          timeZone: "UTC",
          day: "2-digit",
          month: "long",
          year: "numeric",
        })}
      </p>

      <p className="text-slate-600">
        Time: {time}
      </p>

      <p className="text-slate-600">
        Location: {location}
      </p>
    </div>
  );
}

export default EventCard;
```

This is just a display update. Your current Register button still changes local React state only; it does not yet save a registration to MongoDB.

Run both applications

Use two terminals.

Terminal 1 — Express backend

cd E:\event-management-system\server

npm run dev

Terminal 2 — React frontend

cd E:\event-management-system\client

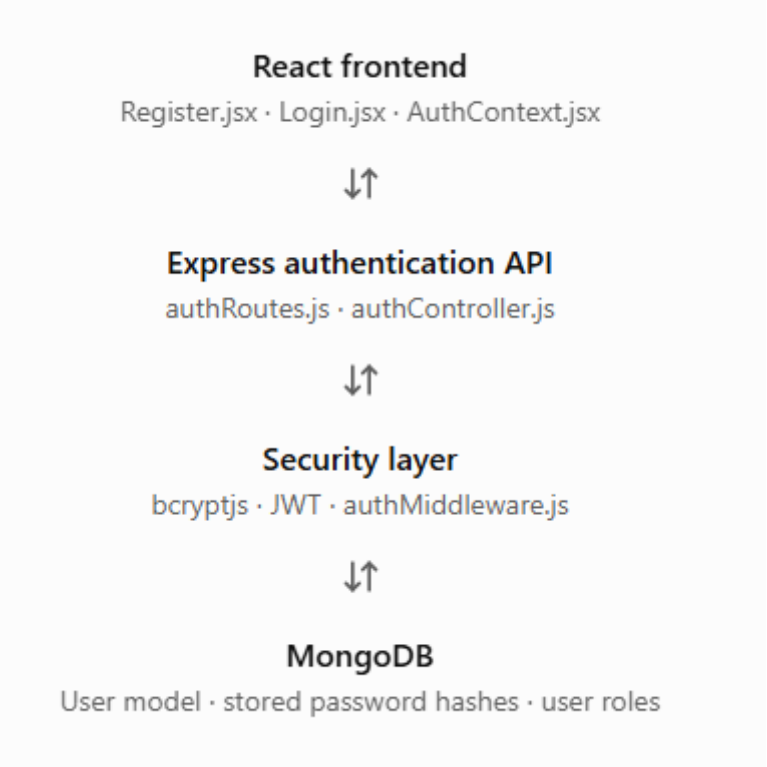
npm run dev

Open the Vite URL printed in the terminal, usually `http://localhost:5173`, and go to `/events`.

You should see the events coming from MongoDB through Express, rather than the hard-coded array.

JWT Authentication

Implementing real user authentication with JWT (JSON Web Tokens) . JWT helps establish the authenticated identity; middleware and role checks enforce authorization. Express middleware is designed to run these checks before a route handler.



JWT login flow

Step 1 — Register

React sends name, email and password to `POST /api/auth/register`. The backend hashes the password and stores the user.

Step 2 — Login

The backend finds the user, compares the submitted password with the stored hash and signs a JWT if the credentials are correct.

Step 3 — Send the token

React sends `Authorization: Bearer YOUR_TOKEN` when calling a protected endpoint.

Step 4 — Verify permissions

Express verifies the token, identifies the user, checks the user's role when required, and then allows or rejects the request.

Configure the JWT secret

Open `server/.env`. Keep your existing MongoDB connection string and add these variables:

```
PORT=5000

MONGODB_URI=YOUR_EXISTING_MONGODB_CONNECTION_STRING

JWT_SECRET=PASTE_A_LONG_RANDOM_SECRET_HERE
JWT_EXPIRES_IN=1d
```

Do not replace your actual `MONGODB_URI` with the example text.

Generate a strong secret using a separate VS Code terminal:

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

Copy the output and use it as the value of `JWT_SECRET`.

Check that `server/.gitignore` contains:

```
node_modules/
.env
```

Never put `JWT_SECRET` in the React frontend. Only the backend can sign and verify your JWTs.

Create the User model

Open `server/models/User.js` and replace its contents with:

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true,
    },

    email: {
      type: String,
      required: [true, "Email is required"],
      unique: true,
      lowercase: true,
      trim: true,
    },

    password: {
      type: String,
      required: [true, "Password is required"],
      select: false,
    },

    role: {
      type: String,
      enum: ["user", "admin"],
      default: "user",
    },
  },
  {
    timestamps: true,
    toJSON: {
      transform: function (doc, ret) {
        delete ret.password;
        delete ret.__v;
        return ret;
      },
    },
  }
);

module.exports = mongoose.model("User", userSchema);
```

Here

- `unique: true` creates a uniqueness constraint for emails; duplicate-email errors must still be handled by the application.
- `lowercase: true` normalizes stored email addresses.
- `select: false` excludes the password hash from ordinary database queries.
- `role` distinguishes a normal user from an administrator.
- `timestamps: true` adds `createdAt` and `updatedAt`.

Notice that a new user defaults to the `user` role. We will not accept a role supplied by the public registration form, because doing so would let someone attempt to register as an admin.

Create the JWT utility

Open `server/utils/generateToken.js`:

```
const jwt = require("jsonwebtoken");

function generateToken(userId) {
  if (!process.env.JWT_SECRET) {
    throw new Error("JWT_SECRET is missing from .env");
  }

  return jwt.sign(
    { userId: userId.toString() },
    process.env.JWT_SECRET,
    {
      algorithm: "HS256",
      expiresIn: process.env.JWT_EXPIRES_IN || "1d",
    }
  );
}

module.exports = generateToken;
```

This generates a token containing the user's ID. The token expires after one day by default. The `jsonwebtoken` library supports token signing, verification and expiration options

Implement registration and login

Open `server/controllers/authController.js` and replace the contents with:

```
const bcrypt = require("bcryptjs");
const User = require("../models/User");
const generateToken = require("../utils/generateToken");
```

```
// POST /api/auth/register
async function registerUser(req, res) {
  try {
    const name =
      typeof req.body.name === "string" ? req.body.name.trim() : "";

    const email =
      typeof req.body.email === "string"
        ? req.body.email.trim().toLowerCase()
        : "";

    const password =
      typeof req.body.password === "string" ? req.body.password : "";

    if (!name || !email || !password) {
      return res.status(400).json({
        message: "Name, email and password are required",
      });
    }

    if (password.length < 6) {
      return res.status(400).json({
        message: "Password must be at least 6 characters",
      });
    }

    if (Buffer.byteLength(password, "utf8") > 72) {
      return res.status(400).json({
        message: "Password is too long",
      });
    }

    const existingUser = await User.findOne({ email });

    if (existingUser) {
      return res.status(409).json({
        message: "An account with this email already exists",
      });
    }

    const hashedPassword = await bcrypt.hash(password, 12);

    // Public registration always creates a normal user.
    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      role: "user",
```

```
    });

    const token = generateToken(user._id);

    return res.status(201).json({
      message: "Registration successful",
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
      },
    });
  } catch (error) {
    // Handles a duplicate-email race condition too.
    if (error.code === 11000) {
      return res.status(409).json({
        message: "An account with this email already exists",
      });
    }

    console.error("Registration error:", error.message);

    return res.status(500).json({
      message: "Registration failed",
    });
  }
}

// POST /api/auth/login
async function loginUser(req, res) {
  try {
    const email =
      typeof req.body.email === "string"
        ? req.body.email.trim().toLowerCase()
        : "";

    const password =
      typeof req.body.password === "string" ? req.body.password : "";

    if (!email || !password) {
      return res.status(400).json({
        message: "Email and password are required",
      });
    }

    // Password is excluded by default, so explicitly select it.
```

```
    const user = await User.findOne({ email }).select("+password");

    if (!user) {
      return res.status(401).json({
        message: "Invalid email or password",
      });
    }

    const passwordMatches = await bcrypt.compare(
      password,
      user.password
    );

    if (!passwordMatches) {
      return res.status(401).json({
        message: "Invalid email or password",
      });
    }

    const token = generateToken(user._id);

    return res.status(200).json({
      message: "Login successful",
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
      },
    });
  } catch (error) {
    console.error("Login error:", error.message);

    return res.status(500).json({
      message: "Login failed",
    });
  }
}

// GET /api/auth/me
// The protect middleware supplies req.user.
function getCurrentUser(req, res) {
  return res.status(200).json({
    user: {
      id: req.user._id,
      name: req.user.name,
      email: req.user.email,
```

```
      role: req.user.role,
    },
  });
}

module.exports = {
  registerUser,
  loginUser,
  getCurrentUser,
};
```

Registration and login now perform real database operations.

For registration, the password passes through:

```
const hashedPassword = await bcrypt.hash(password, 12);
```

For login, the submitted password is compared with the stored hash:

```
const passwordMatches = await bcrypt.compare(
  password,
  user.password
);
```

Do not return the hashed password in API responses.

Implement token verification middleware

Open `server/middleware/authMiddleware.js` and replace its contents with:

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

// Verifies the JWT and loads the current user.
async function protect(req, res, next) {
  try {
    const authorization = req.headers.authorization;

    if (!authorization || !authorization.startsWith("Bearer ")) {
      return res.status(401).json({
        message: "Authentication required. Please log in.",
      });
    }

    const token = authorization.split(" ")[1];
```

```
    if (!token) {
      return res.status(401).json({
        message: "Authentication token is missing",
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET, {
      algorithms: ["HS256"],
    });

    const user = await User.findById(decoded.userId);

    if (!user) {
      return res.status(401).json({
        message: "User no longer exists",
      });
    }

    // The role comes from MongoDB, not from client input.
    req.user = user;

    next();
  } catch (error) {
    if (
      error.name === "JsonWebTokenError" ||
      error.name === "TokenExpiredError" ||
      error.name === "NotBeforeError"
    ) {
      return res.status(401).json({
        message: "Invalid or expired token. Please log in again.",
      });
    }

    console.error("Authentication error:", error.message);

    return res.status(500).json({
      message: "Authentication could not be completed",
    });
  }
}

// Allows only administrators.
function adminOnly(req, res, next) {
  if (!req.user || req.user.role !== "admin") {
    return res.status(403).json({
      message: "Access denied. Admin permission required.",
    });
  }
```

```
  next();
}

module.exports = {
  protect,
  adminOnly,
};
```

The `protect` middleware:

1. Reads the `Authorization` header.
2. Extracts the Bearer token.
3. Verifies its signature and expiry.
4. Retrieves the user from MongoDB.
5. Attaches that user to `req.user`.

The `adminOnly` middleware checks the role from that database user.

Create authentication routes

Open `server/routes/authRoutes.js`:

```
const express = require("express");
const router = express.Router();

const {
  registerUser,
  loginUser,
  getCurrentUser,
} = require("../controllers/authController");

const { protect } = require("../middleware/authMiddleware");

router.post("/register", registerUser);
router.post("/login", loginUser);
router.get("/me", protect, getCurrentUser);

module.exports = router;
```

Authentication API will provide the following endpoints:

Method	Endpoint	Access
POST	`/api/auth/register`	Public
POST	`/api/auth/login`	Public

GET	`/api/auth/me`	Authenticated users

Your five existing event endpoint URLs remain unchanged.

Protect event-management endpoints

Event viewing should remain public. Creating, updating and deleting events should require an administrator.

Open `server/routes/eventRoutes.js` and replace it with:

```
const express = require("express");
const router = express.Router();

const {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent,
} = require("../controllers/eventController");

const {
  protect,
  adminOnly,
} = require("../middleware/authMiddleware");

// Public routes
router.get("/", getEvents);
router.get("/:id", getEventById);

// Admin-only routes
router.post("/", protect, adminOnly, createEvent);
router.put("/:id", protect, adminOnly, updateEvent);
router.delete("/:id", protect, adminOnly, deleteEvent);

module.exports = router;
```

The order matters. For example:

router.post("/", protect, adminOnly, createEvent);

First `protect` authenticates the user. Then `adminOnly` checks the role. Only then does `createEvent` run.

Update the Express server

Open `server/server.js`. Keep the MongoDB connection code you added in the previous stage, but ensure that authentication routes are mounted.

Use this complete version if your database connection function is already in `server/config/db.js`:

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");

const connectDB = require("./config/db");
const eventRoutes = require("./routes/eventRoutes");
const authRoutes = require("./routes/authRoutes");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.status(200).send("Event Management API Running");
});

// Authentication endpoints
app.use("/api/auth", authRoutes);

// Existing event endpoints
app.use("/api/events", eventRoutes);

// Unknown routes
app.use((req, res) => {
  res.status(404).json({
    message: "Route not found",
  });
});

const PORT = process.env.PORT || 5000;

async function startServer() {
  if (!process.env.JWT_SECRET) {
    throw new Error("JWT_SECRET is missing from .env");
  }

  await connectDB();
```

```
  app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
  });
}

startServer().catch((error) => {
  console.error("Server startup failed:", error.message);
  process.exit(1);
});
```

Start your backend:

```
cd E:\event-management-system\server
npm run dev
```

Confirm that the terminal shows both the MongoDB connection message and the server-running message before testing authentication.

Add authentication API functions to React

Open `client/src/services/api.js` and replace its contents with:

```
async function apiRequest(path, options = {}) {
  const token = localStorage.getItem("eventHubToken");

  const headers = {
    ...(options.body ? { "Content-Type": "application/json" } : {}),
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...options.headers,
  };

  const response = await fetch(`/api${path}`, {
    ...options,
    headers,
  });

  const data = await response.json().catch(() => ({}));

  if (!response.ok) {
    throw new Error(data.message || "Request failed");
  }

  return data;
}
```

```
// Existing event API
export function getEvents() {
  return apiRequest("/events");
}

// Register an account
export function registerUser(userData) {
  return apiRequest("/auth/register", {
    method: "POST",
    body: JSON.stringify(userData),
  });
}

// Login
export function loginUser(credentials) {
  return apiRequest("/auth/login", {
    method: "POST",
    body: JSON.stringify(credentials),
  });
}

// Load the current authenticated user
export function getCurrentUser() {
  return apiRequest("/auth/me");
}
```

You can continue importing `getEvents()` in `Events.jsx` without changing its usage.

The important addition is the `Authorization` header. When a token exists, protected requests will be sent with:

Authorization: Bearer eyJ...

The text `eyJ...` is only an illustration; use the actual token returned by your backend.

Security note: This learning implementation uses `localStorage` to keep the workflow understandable and compatible with your existing code. For a production application, OWASP advises against storing session or authentication tokens in localStorage because JavaScript can access them. An `HttpOnly`, `Secure`, appropriately configured `SameSite` cookie is a safer direction

Replace mock AuthContext with real authentication

Open `client/src/context/AuthContext.jsx` and replace its contents with:

```
import {
  createContext,
  useEffect,
```

```
  useState,
} from "react";

import {
  loginUser,
  registerUser,
  getCurrentUser,
} from "../services/api";

export const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [authLoading, setAuthLoading] = useState(true);

  // Restore and verify the login when the app opens.
  useEffect(() => {
    let active = true;

    async function restoreSession() {
      const token = localStorage.getItem("eventHubToken");

      if (!token) {
        setAuthLoading(false);
        return;
      }

      try {
        const data = await getCurrentUser();

        if (active) {
          setUser(data.user);
        }
      } catch {
        localStorage.removeItem("eventHubToken");

        if (active) {
          setUser(null);
        }
      } finally {
        if (active) {
          setAuthLoading(false);
        }
      }
    }

    restoreSession();
```

```
    return () => {
      active = false;
    };
  }, []);

  async function login(email, password) {
    const data = await loginUser({ email, password });

    localStorage.setItem("eventHubToken", data.token);
    setUser(data.user);

    return data;
  }

  async function register(name, email, password) {
    const data = await registerUser({
      name,
      email,
      password,
    });

    localStorage.setItem("eventHubToken", data.token);
    setUser(data.user);

    return data;
  }

  function logout() {
    localStorage.removeItem("eventHubToken");
    setUser(null);
  }

  return (
    <AuthContext.Provider
      value={{
        user,
        authLoading,
        login,
        register,
        logout,
      }}
    >
      {children}
    </AuthContext.Provider>
  );
}

export default AuthProvider;
```

Your `client/src/hooks/useAuth.js` should remain:

```
import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";

function useAuth() {
  return useContext(AuthContext);
}

export default useAuth;
```

Your existing `main.jsx` should already wrap `App` in `AuthProvider`. Keep that setup.

The difference from your previous version is that `login()` now sends credentials to Express instead of constructing a fake user from the email address.

Connect the Register page

Open `client/src/pages/Register.jsx` and replace it with:

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";

import useAuth from "../hooks/useAuth";

function Register() {
  const { register } = useAuth();
  const navigate = useNavigate();

  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
  });

  const [message, setMessage] = useState("");
  const [submitting, setSubmitting] = useState(false);

  function handleChange(event) {
    const { name, value } = event.target;

    setFormData((previous) => ({
      ...previous,
      [name]: value,
    }));
  }
```

```
async function handleSubmit(event) {
  event.preventDefault();
  setMessage("");

  if (
    !formData.name.trim() ||
    !formData.email.trim() ||
    !formData.password
  ) {
    setMessage("Please fill in all fields.");
    return;
  }

  if (formData.password.length < 6) {
    setMessage("Password must be at least 6 characters.");
    return;
  }

  try {
    setSubmitting(true);

    await register(
      formData.name,
      formData.email,
      formData.password
    );

    navigate("/");
  } catch (error) {
    setMessage(error.message || "Registration failed.");
  } finally {
    setSubmitting(false);
  }
}

return (
  <div className="min-h-screen bg-slate-100 px-6 py-16">
    <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">
      <h1 className="text-3xl font-bold text-slate-900">
        Create Account
      </h1>

      <p className="mt-2 text-slate-600">
        Register to start exploring events.
      </p>

      <form
        onSubmit={handleSubmit}
```

```
          className="mt-6 space-y-4"
        >
          <div>
            <label htmlFor="name" className="mb-1 block font-medium">
              Name
            </label>

            <input
              id="name"
              type="text"
              name="name"
              autoComplete="name"
              value={formData.name}
              onChange={handleChange}
              required
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
              placeholder="Enter your name"
            />
          </div>

          <div>
            <label htmlFor="register-email" className="mb-1 block font-medium">
              Email
            </label>

            <input
              id="register-email"
              type="email"
              name="email"
              autoComplete="email"
              value={formData.email}
              onChange={handleChange}
              required
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
              placeholder="Enter your email"
            />
          </div>

          <div>
            <label htmlFor="register-password" className="mb-1 block font-medium">
              Password
            </label>

            <input
              id="register-password"
              type="password"
```

```
            name="password"
            autoComplete="new-password"
            value={formData.password}
            onChange={handleChange}
            minLength={6}
            required
            className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
            placeholder="At least 6 characters"
          />
        </div>

        <button
          type="submit"
          disabled={submitting}
          className="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white transition
hover:bg-indigo-700 disabled:cursor-not-allowed disabled:opacity-60"
        >
          {submitting ? "Creating account..." : "Register"}
        </button>

        {message && (
          <p role="alert" className="text-center text-sm text-red-600">
            {message}
          </p>
        )}
      </form>

      <p className="mt-5 text-center text-sm text-slate-600">
        Already have an account?{" "}
        <Link to="/login" className="font-semibold text-indigo-600">
          Login
        </Link>
      </p>
    </div>
  </div>
 );
}

export default Register;
```

When registration succeeds, the backend returns a token and user information. `AuthContext` saves the session for this learning implementation and updates the navbar. Registration failures, such as a duplicate email, are shown on the page.

Connect the Login page

Open `client/src/pages/Login.jsx` and replace it with:

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";

import useAuth from "../hooks/useAuth";

function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();

  const [formData, setFormData] = useState({
    email: "",
    password: "",
  });

  const [message, setMessage] = useState("");
  const [submitting, setSubmitting] = useState(false);

  function handleChange(event) {
    const { name, value } = event.target;

    setFormData((previous) => ({
      ...previous,
      [name]: value,
    }));
  }

  async function handleSubmit(event) {
    event.preventDefault();
    setMessage("");

    if (!formData.email.trim() || !formData.password) {
      setMessage("Please fill in all fields.");
      return;
    }

    try {
      setSubmitting(true);

      const data = await login(
        formData.email,
        formData.password
      );
```

```
      // Redirect according to the role returned by the backend.
      if (data.user.role === "admin") {
        navigate("/admin");
      } else {
        navigate("/");
      }
    } catch (error) {
      setMessage(error.message || "Login failed.");
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <div className="min-h-screen bg-slate-100 px-6 py-16">
      <div className="mx-auto max-w-md rounded-xl bg-white p-8 shadow-md">
        <h1 className="text-3xl font-bold text-slate-900">
          Login
        </h1>

        <p className="mt-2 text-slate-600">
          Login to manage your event registrations.
        </p>

        <form
          onSubmit={handleSubmit}
          className="mt-6 space-y-4"
        >
          <div>
            <label htmlFor="login-email" className="mb-1 block font-medium">
              Email
            </label>

            <input
              id="login-email"
              type="email"
              name="email"
              autoComplete="email"
              value={formData.email}
              onChange={handleChange}
              required
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
              placeholder="Enter your email"
            />
          </div>

          <div>
```

```
            <label htmlFor="login-password" className="mb-1 block font-medium">
              Password
            </label>

            <input
              id="login-password"
              type="password"
              name="password"
              autoComplete="current-password"
              value={formData.password}
              onChange={handleChange}
              required
              className="w-full rounded-lg border px-4 py-2 outline-none focus:ring-2
focus:ring-indigo-500"
              placeholder="Enter your password"
            />
          </div>

          <button
            type="submit"
            disabled={submitting}
            className="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white transition
hover:bg-indigo-700 disabled:cursor-not-allowed disabled:opacity-60"
          >
            {submitting ? "Logging in..." : "Login"}
          </button>

          {message && (
            <p role="alert" className="text-center text-sm text-red-600">
              {message}
            </p>
          )}
        </form>

        <p className="mt-5 text-center text-sm text-slate-600">
          Don't have an account?{" "}
          <Link to="/register" className="font-semibold text-indigo-600">
            Register
          </Link>
        </p>
      </div>
    </div>
  );
}

export default Login;
```

Now your Login form communicates with the real MongoDB-backed authentication API. An incorrect password will produce an error

Protect the frontend routes

Backend middleware is the actual security boundary. A frontend route guard is still useful because it keeps unauthenticated users from opening protected pages through normal navigation.

Create this new file:

`client/src/components/ProtectedRoute.jsx`

```
import { Navigate, useLocation } from "react-router-dom";

import useAuth from "../hooks/useAuth";
import Loading from "./Loading";

function ProtectedRoute({
  children,
  requireAdmin = false,
}) {
  const { user, authLoading } = useAuth();
  const location = useLocation();

  if (authLoading) {
    return <Loading />;
  }

  if (!user) {
    return (
      <Navigate
        to="/login"
        replace
        state={{ from: location }}
      />
    );
  }

  if (requireAdmin && user.role !== "admin") {
    return <Navigate to="/" replace />;
  }

  return children;
}

export default ProtectedRoute;
```

Now open `client/src/App.jsx` and replace it with:

```
import {
  BrowserRouter,
  Routes,
  Route,
} from "react-router-dom";

import Navbar from "./components/Navbar";
import ProtectedRoute from "./components/ProtectedRoute";

import Home from "./pages/Home";
import Events from "./pages/Events";
import Login from "./pages/Login";
import Register from "./pages/Register";
import MyRegistrations from "./pages/MyRegistrations";
import AdminDashboard from "./pages/AdminDashboard";

function App() {
  return (
    <BrowserRouter>
      <Navbar />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/events" element={<Events />} />

        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        <Route
          path="/my-registrations"
          element={
            <ProtectedRoute>
              <MyRegistrations />
            </ProtectedRoute>
          }
        />

        <Route
          path="/admin"
          element={
            <ProtectedRoute requireAdmin>
              <AdminDashboard />
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
```

```
}

export default App;
```

This makes `MyRegistrations` a signed-in-user page and `/admin` an admin-only page. The current page components are still placeholders for registration data and event administration; this step protects their frontend routes but does not yet implement those features.

Your existing `Navbar.jsx` uses `useAuth()` to read `user` and `logout`, so it should continue working with the new context. You may additionally display an Admin Dashboard link when `user?.role === "admin"`.

Test JWT Authentication using Postman

Start your backend first:

```
cd E:\event-management-system\server
npm run dev
```

Test A: Register a new user

Create a `POST` request:

http://localhost:5000/api/auth/register

Select Body → raw → JSON:

```
{
  "name": "Test User",
  "email": "testuser@example.com",
  "password": "TestPass123"
}
```

Expected status: `201 Created`

The response should contain a token and user information:

```
{
  "message": "Registration successful",
  "token": "your_generated_jwt",
  "user": {
    "id": "generated_mongodb_id",
    "name": "Test User",
    "email": "testuser@example.com",
    "role": "user"
  }
```

```
}
```

The token and ID above are illustrative. Your backend will generate the actual values.

Verify in MongoDB Atlas that the user was created and the `password` field contains a hash, not `TestPass123`.

Test B: Login

Create another `POST` request:

http://localhost:5000/api/auth/login

Body → raw → JSON:

```
{
  "email": "testuser@example.com",
  "password": "TestPass123"
}
```

Expected: `200 OK`, a JWT and user information.

Test with an incorrect password as well. It should return `401 Unauthorized`.

Test C: Verify your token

Create a `GET` request:

http://localhost:5000/api/auth/me

In Postman, open Authorization → Bearer Token, then paste the JWT returned by login.

Expected: `200 OK`, containing the authenticated user's profile.

Remove the token and send the request again. It should return `401 Unauthorized`.

Test D: Verify that event creation is protected

Send a `POST` request to:

http://localhost:5000/api/events

Use a valid event JSON body, but do not send a token.

Expected: `401 Unauthorized`.

Next, send the same request with a normal user's JWT. Expected: `403 Forbidden`.

Public event fetching should continue to work:

GET http://localhost:5000/api/events

This verifies the distinction between authentication and admin authorization.

Create your first admin account

Public registration deliberately creates users with `role: "user"`. For your development project, you can promote a test account manually in MongoDB Atlas.

1. Open Atlas and select your database's `users` collection.
2. Find the test user you registered.
3. Edit that document and change only its `role` field from `"user"` to `"admin"`.
4. Save the document.
5. Log in again with that account.
6. Use the returned token to test `POST`, `PUT` and `DELETE` event endpoints.

Because the middleware reloads the user from MongoDB on each protected request, the database role controls authorization. Logging in again also refreshes the user information shown in React.

Do not add an admin-role selector to the public registration form.

Run the frontend

Open a second terminal:

cd E:\event-management-system\client

npm run dev

Then test this sequence in your browser:

- Frontend test checklist
- Open /register and create an account.
- Confirm that successful registration redirects to Home.
- Check that the navbar reflects the signed-in user.
- Refresh the browser and confirm the session is restored.
- Log out and confirm that My Registrations redirects to Login.
- Log in with incorrect credentials and confirm an error appears.
- Sign in as an admin and open /admin.

26/10/2026 Then after complete

- Event Registration System
  Save user registrations in MongoDB, prevent duplicate registrations, display a user's registrations, and allow the admin to view all registrations.
- Complete the Admin Dashboard
  Implement the interface for creating, editing and deleting events, plus viewing registrations. Connect it to your protected APIs.
- Search, Filter and UI improvements
  Search by event title, filter by location/date, improve form validation and feedback,

and complete the responsive layout. give me code of all files after doing this.

Code here

Socket.IO

Step 1 —Install the dependencies

Run these commands in separate terminals.
Backend:

```
cd server
npm install socket.io
```

Frontend:

```
cd client
npm install socket.io-client
```

Do not remove any existing dependencies.

Step 2 —Modify `server/server.js`

We need to replace `app.listen()` with an HTTP server that Socket.IO can attach to.
Replace your current `server/server.js` with this:

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const http = require("http");
const { Server } = require("socket.io");

const connectDB = require("./config/db");

const authRoutes = require("./routes/authRoutes");
const eventRoutes = require("./routes/eventRoutes");
const registrationRoutes = require("./routes/registrationRoutes");

const {
  notFound,
  errorHandler,
} = require("./middleware/errorMiddleware");
```

```
const app = express();

const allowedOrigin =
  process.env.CLIENT_ORIGIN || "http://localhost:5173";

app.use(cors({ origin: allowedOrigin }));
app.use(express.json({ limit: "100kb" }));

app.get("/", (req, res) => {
  res.status(200).send("Event Management API Running");
});

app.get("/api/health", (req, res) => {
  res.status(200).json({ status: "ok" });
});

app.use("/api/auth", authRoutes);
app.use("/api/events", eventRoutes);
app.use("/api/registrations", registrationRoutes);

app.use(notFound);
app.use(errorHandler);

// Create one HTTP server for Express and Socket.IO
const server = http.createServer(app);

// Attach Socket.IO
const io = new Server(server, {
  cors: {
    origin: allowedOrigin,
    methods: ["GET", "POST"],
  },
});

// Handle client connections
io.on("connection", (socket) => {
  console.log("Socket connected:", socket.id);

  socket.on("disconnect", (reason) => {
    console.log("Socket disconnected:", socket.id, reason);
  });
});

// Make Socket.IO available to Express controllers
app.set("io", io);

const PORT = process.env.PORT || 5000;

async function startServer() {
  if (!process.env.JWT_SECRET) {
    throw new Error("JWT_SECRET is missing from .env");
  }

  await connectDB();
```

```
  server.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
  });
}

startServer().catch((error) => {
  console.error("Server startup failed:", error.message);
  process.exit(1);
});
```

Understand the important changes:

Code	Explanation
`http.createServer(app)`	Creates an HTTP server around Express.
`new Server(server)`	Attaches Socket.IO to the HTTP server.
`io.on("connection")`	Handles new client connections.
`socket.on("disconnect")`	Handles client disconnections.
`app.set("io", io)`	Makes the Socket.IO instance accessible inside controllers.
`server.listen()`	Starts the shared HTTP and Socket.IO server.

Do not keep the old `app.listen()` call. There should only be one server startup.

Step 3 — Test the backend connection

Start the backend:

```
cd server
npm run dev
```

You should see:

Server running at http://localhost:5000

At this point Socket.IO is configured, but it won't log a client connection until we connect the Frontend.

Step 4 — Emit real-time notifications from the event controller

Now we need to tell connected clients when an admin creates, updates, or deletes an event.

Open:

`server/controllers/eventController.js`

We will use the `io` instance that we stored using `app.set("io", io)`.

4.1 Modify `createEvent`

Find the end of your existing `createEvent` function:

```
const event = await Event.create({
  ...eventPayload(req.body),
  createdBy: req.user._id,
});

res.status(201).json({
  message: "Event created successfully",
  event,
```

```
});
```
Replace that section with:
```
const event = await Event.create({
  ...eventPayload(req.body),
  createdBy: req.user._id,
});

const io = req.app.get("io");

io.emit("event:created", {
  message: `A new event has been added: ${event.title}`,
  event: {
    _id: event._id,
    title: event.title,
    date: event.date,
    location: event.location,
  },
  createdAt: new Date(),
});

res.status(201).json({
  message: "Event created successfully",
  event,
});
```

4.2 Modify `updateEvent`

Find the successful update section:
```
if (!event) {
  return res.status(404).json({ message: "Event not found" });
}

res.status(200).json({
  message: "Event updated successfully",
  event,
});
```
Replace it with:
```
if (!event) {
  return res.status(404).json({ message: "Event not found" });
}

const io = req.app.get("io");

io.emit("event:updated", {
  message: `An event has been updated: ${event.title}`,
  event: {
    _id: event._id,
    title: event.title,
    date: event.date,
    location: event.location,
  },
  createdAt: new Date(),
});

res.status(200).json({
```

```
  message: "Event updated successfully",
  event,
});
```

4.3 Modify `deleteEvent`

Find the successful deletion section:

```
await Registration.deleteMany({ event: event._id });

res.status(200).json({
  message: "Event deleted successfully",
  event,
});
```

Replace it with:

```
await Registration.deleteMany({ event: event._id });

const io = req.app.get("io");

io.emit("event:deleted", {
  message: `An event has been cancelled: ${event.title}`,
  event: {
    _id: event._id,
    title: event.title,
  },
  createdAt: new Date(),
});

res.status(200).json({
  message: "Event deleted successfully",
  event,
});
```

We use `event:deleted` as the technical event name because your existing API deletes events. Its user-facing notification says the event was cancelled. If you later implement a separate cancellation status instead of deletion, we should introduce a distinct `event:cancelled` event.
Why do we emit only after the database operation succeeds?
Because users should never receive a notification for an event that wasn't successfully saved, updated, or deleted.

Step 5 — Create the React Socket.IO notification component

Create a new file:

`client/src/components/NotificationBell.jsx`

This component will:

- Connect to the backend.
- Listen for all three event notifications.
- Store notifications in React state.
- Display a notification count.
- Open a notification dropdown.
- Disconnect and remove listeners when unmounted.

Add this complete code:

```
import { useEffect, useState } from "react";
import { io } from "socket.io-client";
```

```
const SOCKET_URL =
  import.meta.env.VITE_SOCKET_URL || "http://localhost:5000";

export default function NotificationBell() {
  const [notifications, setNotifications] = useState([]);
  const [isOpen, setIsOpen] = useState(false);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    const socket = io(SOCKET_URL);

    function handleConnect() {
      setConnected(true);
      console.log("Connected to EventHub notifications");
    }

    function handleDisconnect() {
      setConnected(false);
      console.log("Disconnected from EventHub notifications");
    }

    function addNotification(notification) {
      setNotifications((previous) => [
        {
          ...notification,
          id: `Date.now() −{Math.random()}`,
        },
        ...previous,
      ]);
    }

    socket.on("connect", handleConnect);
    socket.on("disconnect", handleDisconnect);

    socket.on("event:created", addNotification);
    socket.on("event:updated", addNotification);
    socket.on("event:deleted", addNotification);

    return () => {
      socket.off("connect", handleConnect);
      socket.off("disconnect", handleDisconnect);

      socket.off("event:created", addNotification);
      socket.off("event:updated", addNotification);
      socket.off("event:deleted", addNotification);

      socket.disconnect();
    };
  }, []);

  function clearNotifications() {
    setNotifications([]);
  }
```

```
  return (
    <div className="relative">
      <button
        type="button"
        onClick={() => setIsOpen((open) => !open)}
        aria-label="Notifications"
        aria-expanded={isOpen}
        className="relative rounded-lg px-3 py-2 text-white transition hover:bg-slate-700"
      >
        <span className="text-xl" aria-hidden="true">🔔</span>

        {notifications.length > 0 && (
          <span className="absolute -right-1 -top-1 flex h-5 min-w-5 items-center justify-center
rounded-full bg-red-600 px-1 text-xs font-bold text-white">
            {notifications.length > 99 ? "99+" : notifications.length}
          </span>
        )}
      </button>

      {isOpen && (
        <div className="absolute right-0 z-50 mt-3 w-80 max-w-[90vw] rounded-xl border
border-slate-200 bg-white p-4 text-slate-900 shadow-xl">
          <div className="flex items-center justify-between gap-2">
            <h2 className="font-bold">Notifications</h2>

            <button
              type="button"
              onClick={clearNotifications}
              className="text-xs font-semibold text-indigo-600 hover:text-indigo-800"
            >
              Clear all
            </button>
          </div>

          <p className="mt-1 text-xs text-slate-500">
            Connection: {connected ? "Connected" : "Disconnected"}
          </p>

          <div className="mt-3 max-h-80 space-y-3 overflow-y-auto">
            {notifications.length === 0 ? (
              <p className="py-6 text-center text-sm text-slate-500">
                No notifications yet.
              </p>
            ) : (
              notifications.map((notification) => (
                <div
                  key={notification.id}
                  className="rounded-lg bg-slate-50 p-3"
                >
                  <p className="text-sm font-medium">
                    {notification.message}
                  </p>

                  <p className="mt-1 text-xs text-slate-400">
                    {new Date(notification.createdAt).toLocaleString()}
```

```
            </p>
          </div>
        ))
      )}
    </div>
  </div>
 )}
</div>
);
}
```

How the component works:

React/Socket.IO code	Purpose
`io(SOCKET_URL)`	Establishes the client connection.
`socket.on()`	Listens for server events.
`setNotifications()`	Adds received notifications to the UI.
`socket.off()`	Removes event listeners.
`socket.disconnect()`	Closes the connection on cleanup.

The `useEffect` cleanup is important. It prevents duplicate listeners and connections when React components are unmounted or remounted.

Step 6 — Add the notification bell to your Navbar

Open:
`client/src/components/Navbar.jsx`

6.1 Add this import

At the top, add:
import NotificationBell from "./NotificationBell.jsx";
Your imports should now look like:

```
import { useState } from "react";
import { Link, NavLink } from "react-router-dom";
import useAuth from "../hooks/useAuth.js";
import NotificationBell from "./NotificationBell.jsx";
```

6.2 Add the notification bell to the navigation

Find this existing section in your Navbar:

```
<button
  type="button"
  onClick={() => setMenuOpen((open) => !open)}
  aria-label={menuOpen ? "Close menu" : "Open menu"}
  aria-expanded={menuOpen}
  className="rounded-lg border border-slate-600 px-3 py-2 md:hidden"
>
  {menuOpen ? "✕" : "☰"}
</button>
```

Place the notification component immediately before that button:

```
<NotificationBell />
```

```
<button
  type="button"
  onClick={() => setMenuOpen((open) => !open)}
  aria-label={menuOpen ? "Close menu" : "Open menu"}
  aria-expanded={menuOpen}
  className="rounded-lg border border-slate-600 px-3 py-2 md:hidden"
>
  {menuOpen ? "✕" : "☰"}
</button>
```

This displays the notification bell on desktop and mobile.

Step 7 — Configure the frontend environment variable

In the `client` folder, create or update:

`client/.env`

```
VITE_SOCKET_URL=http://localhost:5000
```

Your backend `.env` should already have its existing configuration, including:

```
PORT=5000
CLIENT_ORIGIN=http://localhost:5173
```

Keep your existing MongoDB URI and JWT secret. Do not share those values publicly.
Restart the Vite development server after creating or changing `.env`.

Step 8 — Test the complete real-time feature

Start your backend:

```
cd server
npm run dev
```

Start your frontend:

```
cd client
npm run dev
```

Testing setup
Browser 1 — Admin
Log in using an admin account and open the Admin Dashboard.
Browser 2 — Normal user
Open a separate browser or an incognito window and log in using a normal user account. Open EventHub and keep the page open.

Test 1 — Create an event

1. In Browser 1, create an event.
2. Save it successfully.
3. Look at Browser 2.
4. Click the notification bell.

Expected notification:
A new event has been added: Tech Fest 2026

Test 2 — Update an event

1. In Browser 1, edit the event.
2. Save the changes.
3. Look at Browser 2.

Expected notification:

An event has been updated: Tech Fest 2026

Test 3 — Delete an event

1. In Browser 1, delete an event.
2. Confirm deletion.
3. Look at Browser 2.

Expected notification:
An event has been cancelled: Tech Fest 2026

Test 4 — Connection and disconnection

1. Open the browser developer console.
2. Refresh the page.
3. Check the backend terminal.
4. Close the user browser tab.
5. Check the backend terminal again.

Expected backend messages:
Socket connected: <socket-id>
Socket disconnected: <socket-id> transport close
The exact disconnect reason may differ.

Step 9 — Run your project checks

In the frontend terminal:
npm run lint
npm run build
In the backend terminal, check that the server starts and that these still work:

- `GET http://localhost:5000/api/health`
- `GET http://localhost:5000/api/events`

Files modified and created

File	Action
`server/package.json`	Socket.IO dependency added
`client/package.json`	Socket.IO client dependency added
`server/server.js`	Modified
`server/controllers/eventController.js`	Modified
`client/src/components/NotificationBell.jsx`	New
`client/src/components/Navbar.jsx`	Modified
`client/.env`	New or updated

Feature target
Real-time event notifications, connection handling, disconnection handling, and React listener cleanup.

2/11/2026

Code So far

Docker, DevOps & Deployment